BCSE301L_SOFTWARE ENGINEERING

Presented By,

Dr.Baiju B V

Assistant Professor

School of Computer Science and Engineering

VIT, Vellore

DESIGN CONCEPTS AND PRINCIPLES

Software Design

- Software design deals with ***transforming the customer requirements***, as described by the SRS document, into a form that is ***implementable*** using a programming language.
- For a design to be easily implementable in a conventional programming language, the following items must be designed during the design phase.
  - ***Different modules*** *required to implement the design solution.*
  - ***Control relationships*** *among the identified modules. The relationship is also known as the call relationship or invocation relationship among modules.*
  - ***Interface*** *among different modules. The interface among different modules identifies the exact data items exchanged among the modules.*
  - ***Data structures*** *of the individual modules.*
  - ***Algorithms*** *required to implement the individual modules.*
- The objective of the design phase is to take the SRS document as the input and produce the above mentioned documents before completion of the design phase.

- We can broadly classified the design activities into two important parts:

a) Preliminary (or High level) Design

- High level design means ***identification of different modules***, the ***control relationships*** and the ***definitions of the interfaces*** among them.
- The outcome of the high level design is called the ***program structure*** or ***software architecture***.
- Many different types of notations have been used to represent a high level design.
- A popular way is to use a tree like diagram called the ***structure chart*** to represent the control hierarchy in a high level design.

b) Detailed Design

- During detailed design, the ***data structure*** and the ***algorithms*** of different modules are designed.
- The outcome of the detailed design stage is usually known as the ***module specification document***.

High-Level Design of a Hospital Management System

1. Identified Modules:

a. **Patient Management Module:** Handles patient registration, appointment scheduling, and medical history.

b. **Doctor Management Module:** Manages doctor profiles, specialization, availability, and assigned patients.

c. **Appointment & Scheduling Module**

d. **Billing & Payment Module**

2. Control Relationships:

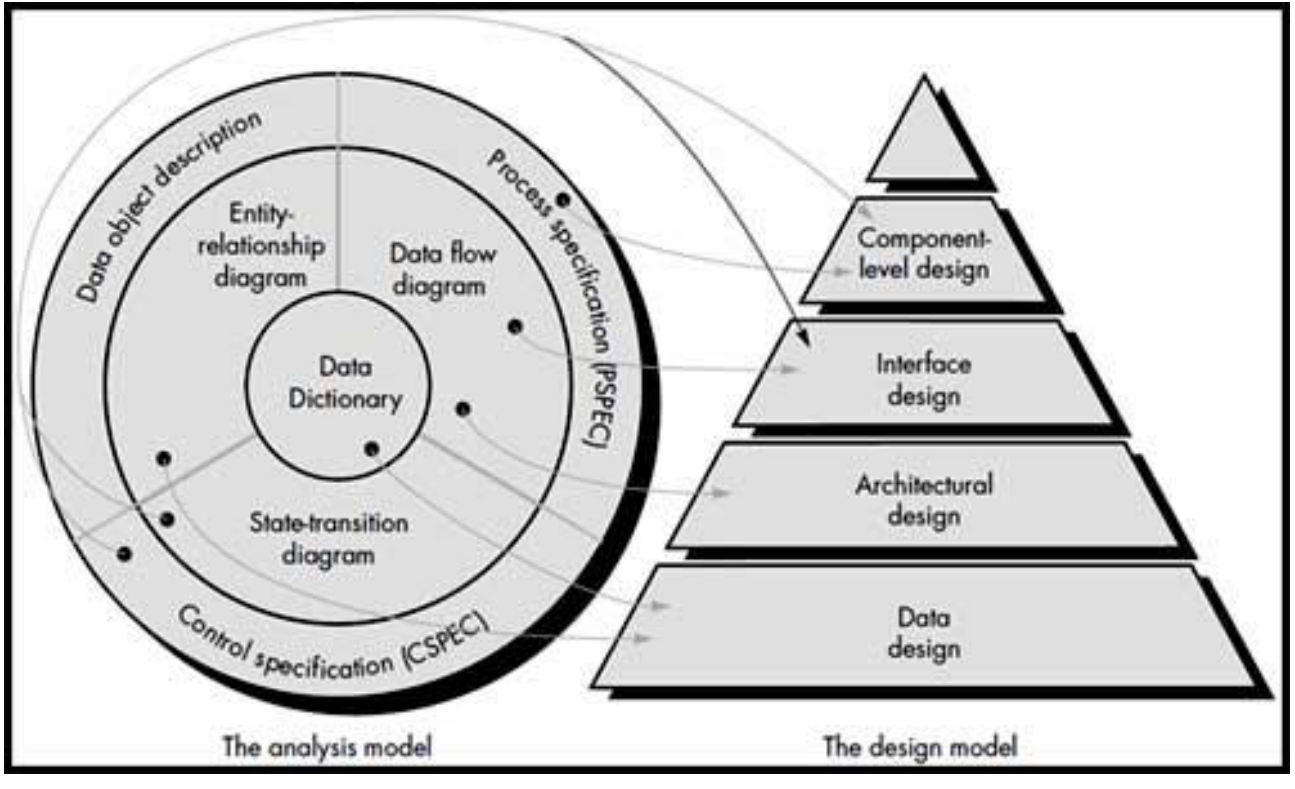
- The **Patient Management Module** interacts with **Doctor Management** and **Appointment Scheduling** to assign patients to doctors.
- The **Billing Module** interacts with **Pharmacy** and **Laboratory** for generating invoices.

3. Interfaces Between Modules:

- **Patient ↔ Doctor**: Patients book appointments through the scheduling module, and doctors access patient records.
- **Doctor ↔ Laboratory**: Doctors request tests, and the lab updates reports in the patient's history.
- **Patient ↔ Billing**: Patients receive and pay for medical services.

Design Model

- The design principles and concepts establish a foundation for the creation of the design model that encompasses ***representation of data***, ***architecture***, ***interface*** and ***components***.
- Each of these design representations is tied to the others, and all can be traced back to software requirements
- The ***Entity-Relationship Diagrams*** (ERD), the ***Data Flow Diagrams*** (DFD), the ***State Transition Diagrams*** (STD) and the ***Data Dictionaries*** (DD) that are constructed during the requirements phase are directly mapped onto the corresponding design model.



Data / Class Design	• Transforms information domain model into ***data structures*** • The data objects (or entities) and the relationships defined in ER diagram and the detailed data content depicted in the Data Dictionary provide the basis for the data design activity.
Architectural Design	• Defines relationship between structural elements of the software, architectural style, and patterns. • Represents the ***framework of computer based systems***.
Interface Design	• Explains how the software interacts with other systems and the people who use it. • An interface implies a flow of information (e.g., data and/or control) and a specific type of behavior.
Component-level Design	• Converts the software architecture into a step-by-step description of how each component works.

- The importance of software design can be stated with a single word – **Quality**.
- Design provides us with representations of software that can be accessed for quality.
- Design is the only way that we can accurately translate a customer's requirements into a finished software product or system.
- Software design serves as the foundation for all the software engineering and software support activities

Design Process

- Software design is an ***iterative process*** through which requirements are translated into a "blueprint" for constructing the software.
- McGlaughlin suggests **3 characteristics** that serve as a guide for evaluation of a good design:

  1. The design must ***implement all of the explicit requirements*** contained in the analysis model and it must accommodate all of the implicit requirements desired by the customer.

  2. The design must be ***readable***, ***understandable guide*** for those who generate code and for those who test and support the software.

  3. The design should provide a ***complete picture of the software***, addressing all data, functional and behavioral domains.

Quality Guidelines.

- To evaluate the quality of a design representation, you and other members of the software team must establish technical criteria for good design.

1. A design should exhibit an ***architecture*** that

(a) has been created using recognizable architectural styles or patterns

(b) is composed of components that exhibit good design characteristics

(c) can be implemented in an evolutionary fashion thereby facilitating implementation and testing.

2. A design should be ***modular***; that is, the software should be logically partitioned into elements or subsystems.

3. A design should contain ***distinct representations of data***, ***architecture***, ***interfaces***, and ***components***.

Data : Stores patient details, doctor schedules, and medical records.

Architecture: A web-based system with a database, server, and user interface.

Interfaces: Patients book appointments through a web portal; doctors access reports via a dashboard.

Components: Patient management, appointment scheduling, billing, and pharmacy modules.

4. A design should ***lead to data structures*** that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.

Patient Records: Use a database table with fields like name, age, medical history.

Doctor Schedule: Use a hash map or calendar structure for quick access.

Billing System: Use a queue to process payments in order.

Medical Reports: Use a tree structure to organize reports by date and category.

5. A design should lead to ***components that exhibit independent functional characteristics***.

6. A design should ***lead to interfaces*** that reduce the complexity of connections between components and with the external environment.

7. A design should be ***derived using a repeatable method*** that is driven by information obtained during software requirements analysis.

8. A design should be ***represented using a notation*** that effectively communicates its meaning.

Design Principles

- Software design is both a process and a model.
- **Design process** : A sequence of steps that enable the designer to describe all aspects of the software to be built.
- **Design model :** An equivalent of an architect's plan for a house. It begins by representing the totality of the thing to be built (e.g. a 3D house) and slowly refining it into more details.
- Similarly, the design model that is created for software provides a variety of different views of the computer software.

Davis – Design Principles

1. The design process ***should not suffer from "tunnel vision".***

- A good designer should consider alternative approaches, judging each based on the requirements of the problem, and the resources available to do the job.

2. The design should be ***traceable to the analysis model***.

- It is necessary to have a means for tracking how requirements have been satisfied by the design model.

3. The designer ***should not reinvent the wheel***

- Use the set of design patterns, already encountered so that new patterns are not reinvented.
- Time is short and resources are limited.
- Design time should be invested in representing truly new ideas and integrating those patterns that already exist.

4. The design should "***minimize the intellectual distance***" between the software and the problem as it exists in the real world

- The structure of the software design should mimic the structure of the problem domain.

5. The design should ***exhibit uniformity and integration***.

- A design is uniform if it appears that one person developed the entire thing.
- Rules of style and format should be defined for a design team before design work begins.
- A design is integrated if care is taken in defining interfaces between design components.

6. The design should be ***structured to accommodate change***.

7. The design should be ***structured to degrade gently***, even when abnormal data, events or operating conditions are encountered.

- Well designed software should never "bomb".
- It should be designed to accommodate unusual circumstances and if it must terminate processing, do so in a graceful manner.

8. Design is not coding, coding is not design.

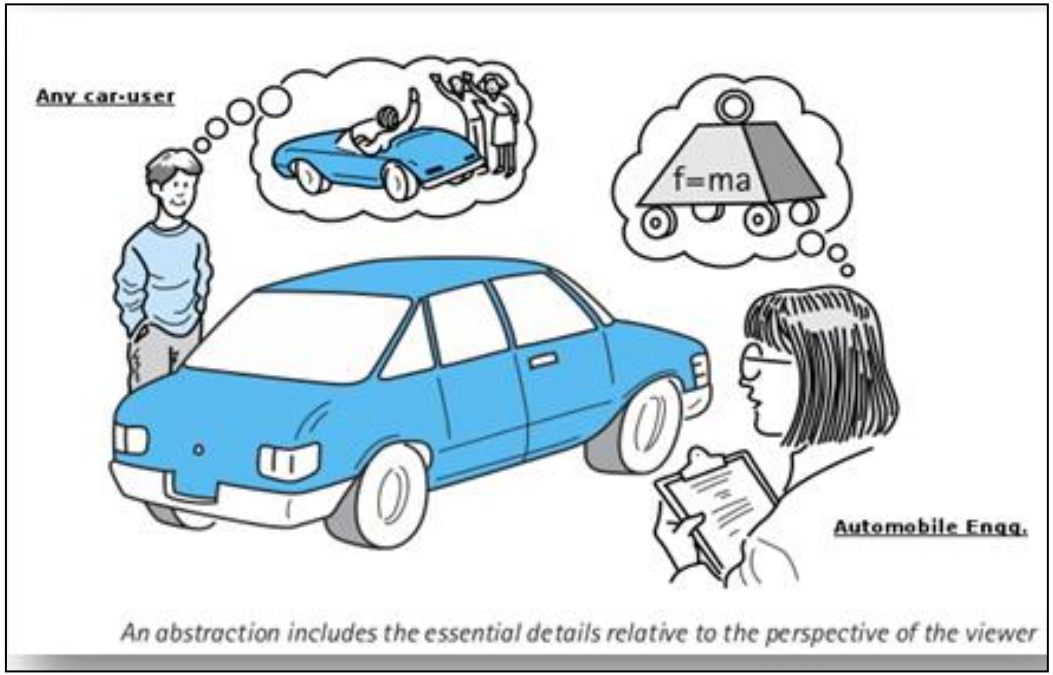
9. The design ***should be assessed for quality*** as it is being created, not after the fact.

10. The design should be ***reviewed to minimize conceptual (semantic) errors***.

Design Concepts

1. Abstraction (Hide Irrelevant data)

- Abstraction is used to ***hide background details or necessary implementation about the data***.
- At the **highest level of abstraction**, it describes the system broadly using terms familiar to the problem domain.
- At **lower levels of abstraction**, a more procedural orientation is taken.



An abstraction includes the essential details relative to the perspective of the viewer

High-level abstraction: The system will manage patient records, appointments, billing, and pharmacy.

Lower-level details: The system will use a SQL database for patient records and an API for appointment booking.

Procedural Abstraction

- Refers to a **sequence of instructions that have a specific and limited function**.
- The name of a procedural abstraction implies these functions, but specific details are suppressed.
- Example : Word **open for a door**.
  - Open implies a long sequence of procedural steps
    - walk to the door,
    - reach out and grasp knob
    - turn knob and pull door
    - step away from moving door

Data Abstraction

- **Named collection of data that describes a data object**
- Example: The data abstraction for door would encompass a set of attributes that describe the door
  - Door type
  - Swing direction
  - Opening mechanism
  - Weight
  - Dimensions

2. Architecture (Design a structure of something)

- Software architecture refers to "the ***overall structure of the software and the ways in which that structure provides conceptual integrity for a system***"
- Architecture is the structure or organization of program components (modules), the ways in which these components interact, and the structure of data that are used by the components.
- **Structural properties** define "the ***components of a system*** (e.g., modules, objects, filters) and the ***manner in which those components are packaged and interact with one another***."
- **Extra-functional properties** address "how the design architecture achieves requirements for
  - Performance
  - Capacity
  - Reliability
  - Security
  - Adaptability
  - their system characteristics (e.g., nonfunctional system requirements)

- An architectural design can be represented using any of the 5 models

Model	Description
Structural Models	• Represent architecture as an ***organized collection of program components***.
Framework Models	• Identify repeatable architectural design framework that similar to the types of applications • ***Client-Server Architecture, Event-Driven Architecture, Model-View-Controller***
Dynamic Models	• Address the behavioral aspects of the program architecture which define how the system responds to external events, such as user inputs, sensor data, or network requests.
Process Models	• Focus design of the business or technical process of a system that the system must accommodate.
Functional Models	• Can be used to represent the functional hierarchy of a system. • It breaks down the system into smaller functions or modules, showing how they interact.

- A number of different **Architectural Description Languages** (ADLs) have been developed to represent these models

3. Patterns (a Repeated form)

- A pattern simply means a ***repeated form or design in which the same shape is repeated several time to form a pattern***.
- A design pattern is a reusable solution to a common design problem.
- It provides a structured way to solve issues within a specific context while considering factors that may affect how the solution is applied.
- The intent of each design pattern is to provide a description that enables a designer to determine

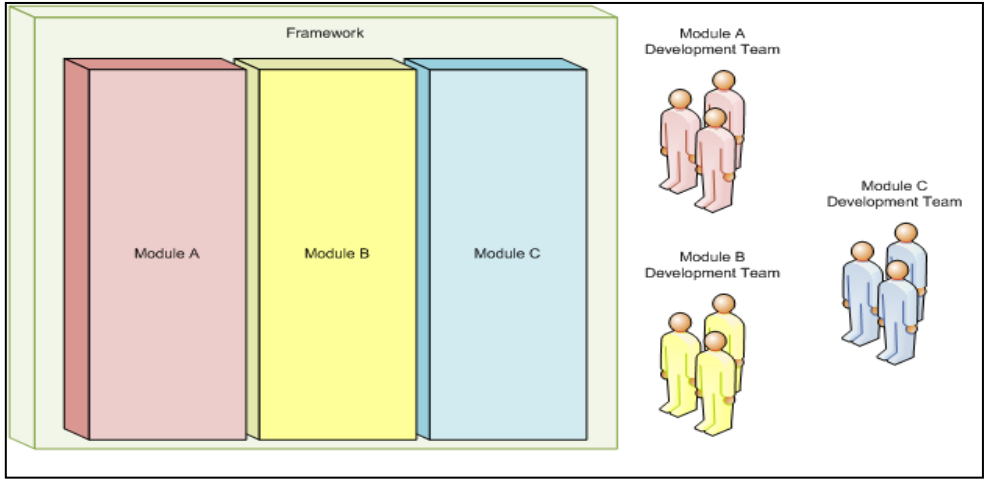
  (1) Whether the ***pattern is applicable to the current work.***

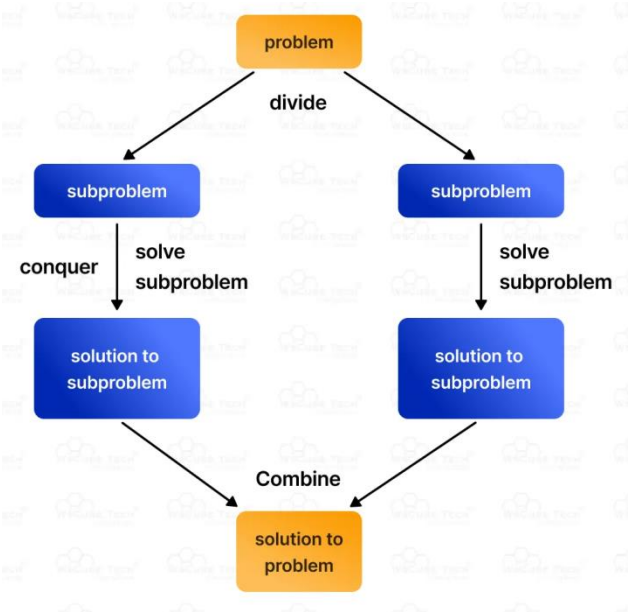
  (2) Whether the ***pattern can be reused*** (hence, saving design time),

  (3) Whether the ***pattern can serve as a guide for developing a similar, but functionally or structurally different, pattern.***

4. Separation of Concerns

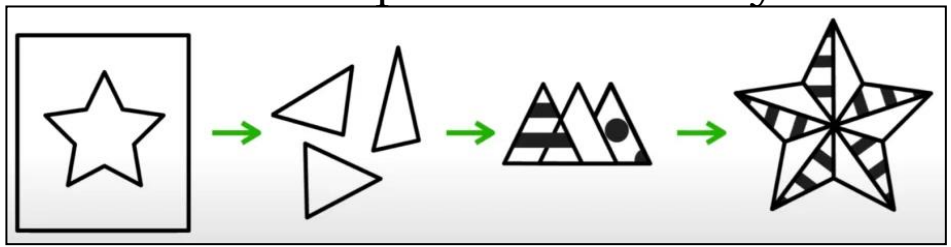
- Separation of concerns is a design concept that suggests that ***any complex problem can be more easily handled if it is subdivided into pieces*** that can each be solved and/or optimized independently.
- A **concern** is a feature or behavior that is specified as part of the requirements model for the software.
- Leads to a ***divide-and-conquer strategy*** —it's easier to solve a complex problem when you break it into manageable pieces.
- Separation of concerns is showed in other related design concepts:
  - **Modularity**
  - **Functional independence**
  - **Refinement**



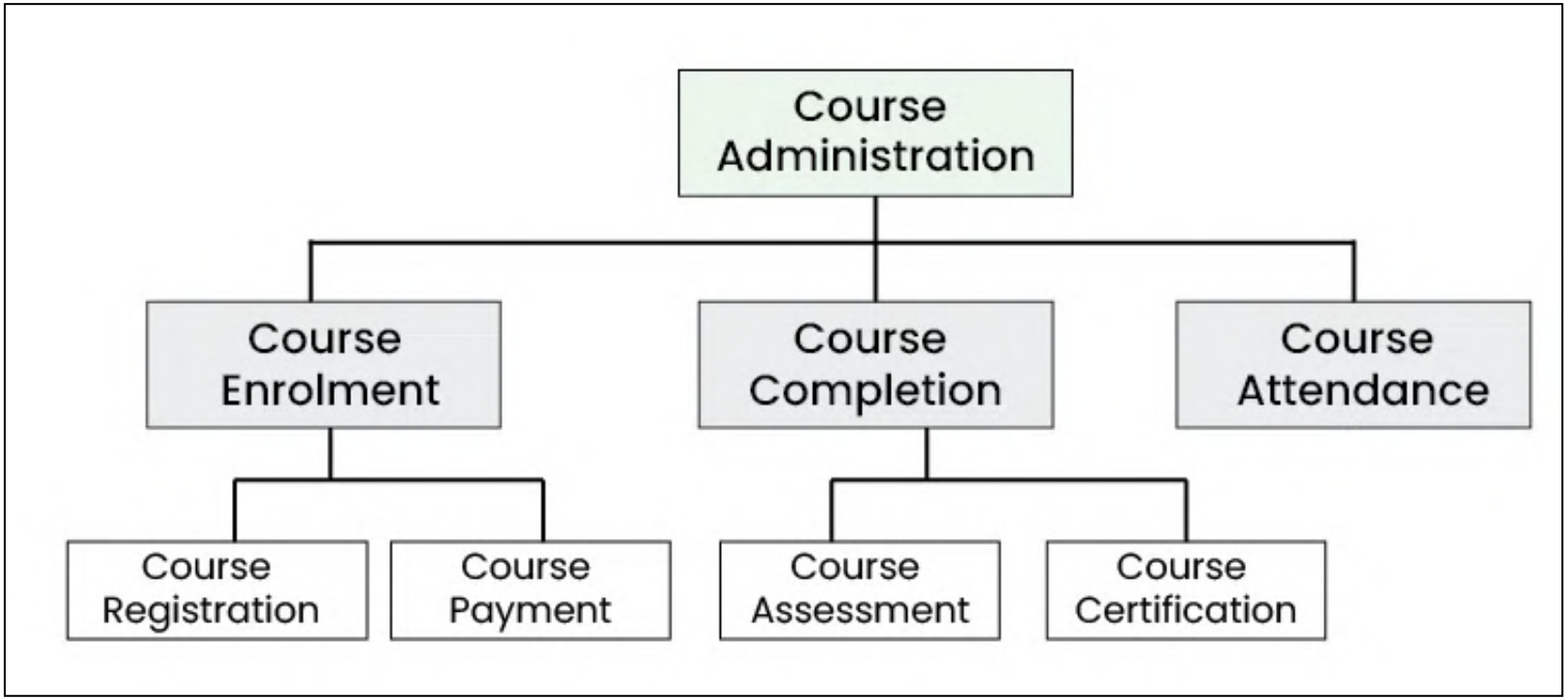


5. Modularity (Subdivide the system)

- Modularity means ***dividing the system or project into smaller parts*** to reduce the complexity of the system or project.
- **5 criteria** to evaluate a design method with respect to its ability to define effective modular system.



Modular Decomposability	Provides a systematic approach for decomposing the problem into subproblems
Modular Composability	Enables existing(reusable) design components to be assembled into a new system
Modular Understandability	Module can be understood as a standalone unit (no need to refer to other modules)
Modular Continuity	Small changes to the system requirements result in changes to individual modules
Modular Protection	Unexpected condition occurs within a module & its effects are constrained within that module


Course Administration
Course Enrolment
Course Completion
Course Attendance
Course Registration
Course Payment
Course Assessment
Course Certification

6. Information Hiding

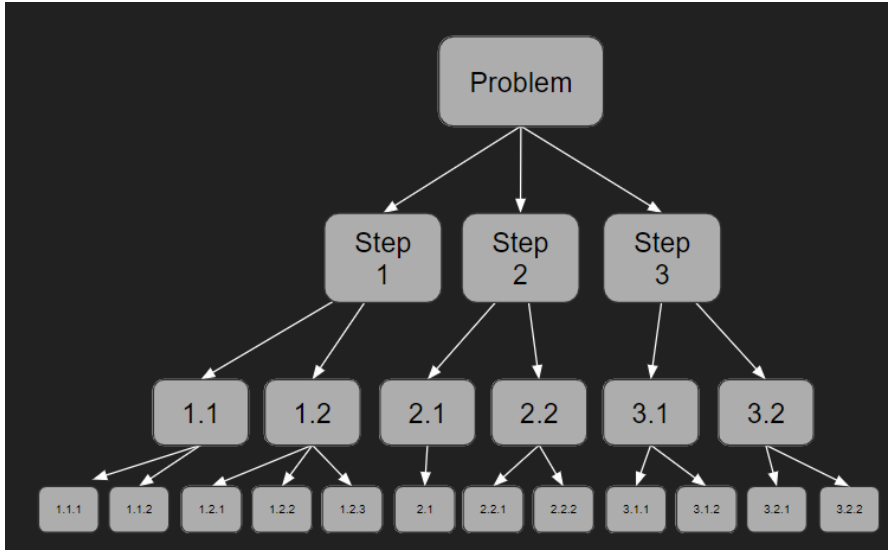
- Modules should be specified and designed so that information (algorithms and data) contained within a module is inaccessible to other modules that have no need for such information.
- Hiding implies that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve software function.
- Abstraction helps to define the procedural entities that make up the software.
- Hiding defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module.

7. Functional Independence

- Functional independence means that each module in a system performs a specific task without relying too much on other modules.
- It comes from key software principles separation of concerns, modularity, and the concepts of abstraction and information hiding
- Independence is assessed using two qualitative criteria: cohesion and coupling.
  - ***Cohesion***: An indication of the relative ***functional strength of a module***.
  - ***Coupling***:An indication of the relative ***interdependence among modules***.

8. Stepwise Refinement

- Stepwise refinement is a top-down design strategy originally proposed by Niklaus Wirth.
- A program is developed by successively ***refining levels of procedural detail***.
- A hierarchy is created by breaking down a big task (procedural abstraction) into smaller steps, continuing this process until it reaches actual programming statements.
- Refinement causes the designer to elaborate on the original statement, providing more and more detail as each successive refinement (elaboration)occurs



9. Refactoring

- Refactoring is a way to improve the design or code of a component without changing how it works.
- When software is refactored, the existing design is examined for
  - Redundancy
  - Unused design elements
  - Inefficient or unnecessary algorithms
  - Poorly constructed or inappropriate data structures
  - Any other design failure that can be corrected to yield a better design.

```c
#include <stdio.h>
int main() {
  int x, y, z;
  x = 10;
  y = 20;
  z = x + y;
  printf("The sum is: %d\n", z);
  return 0;
}
```

```c
#include <stdio.h>
int main() {
  int firstNumber = 10;
  int secondNumber = 20;
  int sum = firstNumber + secondNumber;
  printf("The sum is: %d\n", sum);
  return 0;
}
```

10. Design Classes

- Design classes refine analysis classes by adding details needed for implementation and building the software structure that supports the business solution.
- They define **four characteristics** of a well-formed design class:

(i) Complete and Sufficient.

- A design class should include all the necessary attributes and methods that logically belong to it based on its name.

(ii) Primitiveness.

- Each method in a design class should perform only one specific task.
- Once a method is created for a task, the class should not have another method doing the same thing.

(iii) High cohesion

- A well-designed class has a clear, focused job and uses its attributes and methods only for that purpose.

(iv) Low coupling

- In design, classes should work together, but only as much as necessary to keep things efficient and manageable.
- If a design model is highly coupled (all design classes collaborate with all other design classes), the system is difficult to implement, to test, and to maintain over time.

ABSTRACTION

- Abstraction is used to ***hide background details or necessary implementation about the data***.
- Abstraction can be used for existing element as well as the component being designed.

Procedural abstraction

- Refers to a ***sequence of instructions that have a specific and limited function***.
- The name of a procedural abstraction implies these functions, but specific details are suppressed.

<u>Example</u>: The word ***use*** for a camera in the **SafeHome system**.

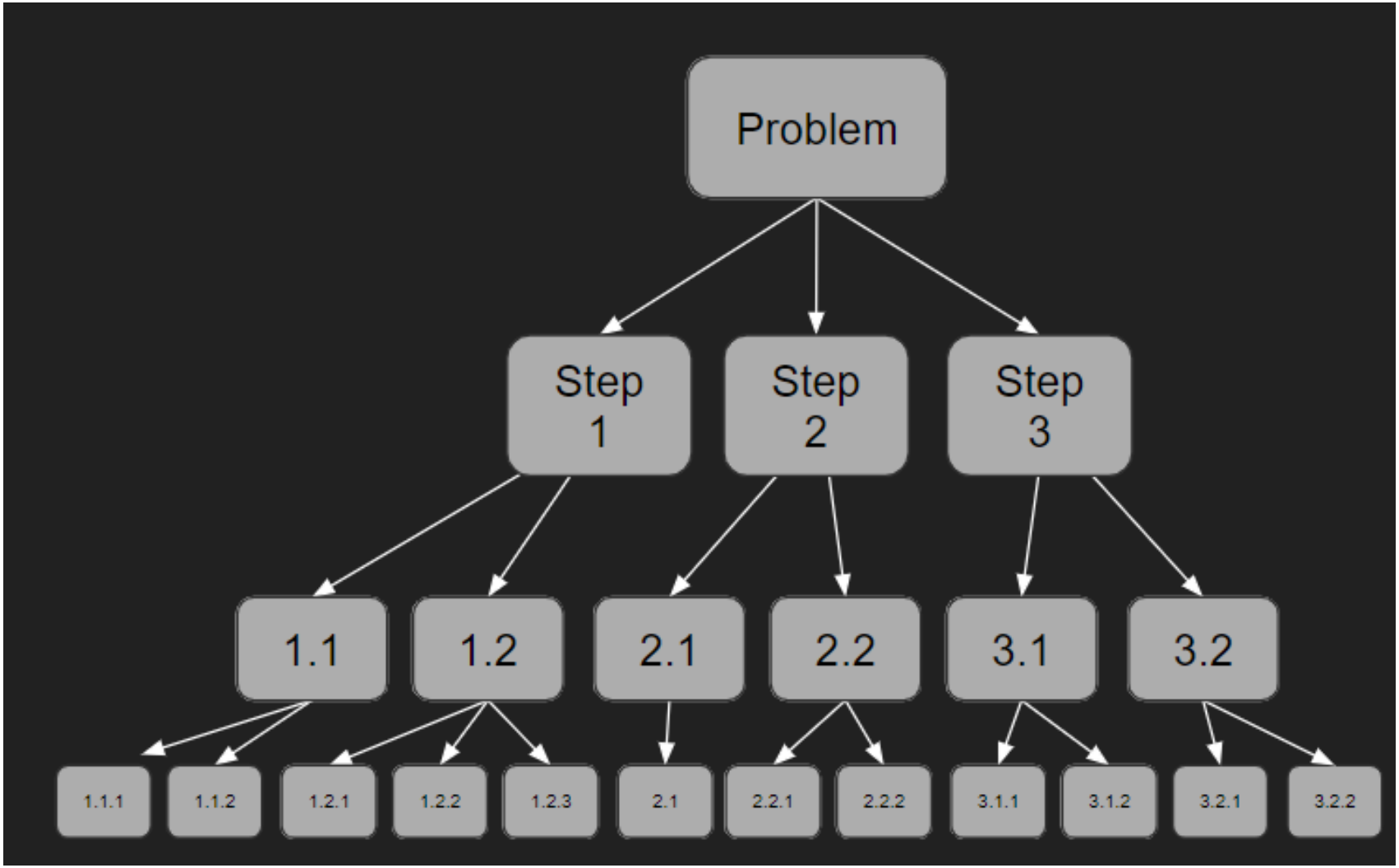
- **Use** implies a long sequence of procedural steps
  - ✓ activate the SafeHome system on a mobile device
  - ✓ log on to the SafeHome system
  - ✓ select a camera to preview
  - ✓ locate the camera controls on mobile app user interface

Data abstraction

- ***Named collection of data that describes a data object***
- In the context of the procedural abstraction *open*, we can define a data abstraction called **camera.**
- The data abstraction for **camera** would encompass a set of attributes that describe the camera
  - camera ID
  - Location
  - field view
  - Pan angle
  - zoom
- It follows that the procedural abstraction *use* would make use of information contained in the attributes of the data abstraction **camera.**

REFINEMENT

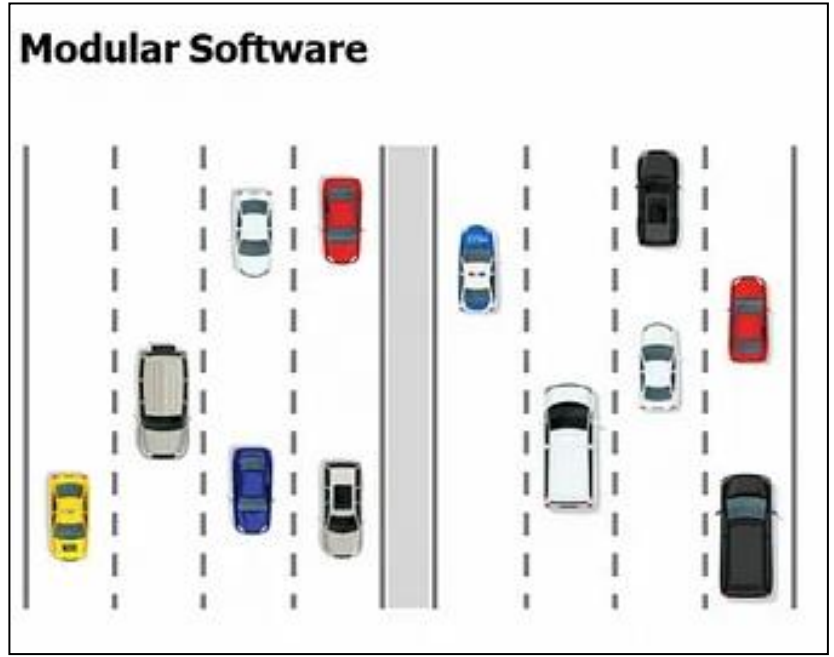
- Refinement means ***removing any impurities and improving the quality of something***.
- The software design refinement idea is a process of building or presenting the software or system in a detailed manner, which implies elaborating on a system or software.
- Refinement is ***essential for identifying and correcting any possible errors***.
- A hierarchy is established by decomposing a statement of function in a stepwise manner till the programming language statement are reached.
- Stepwise refinement involve the following activities :
  - ***Decomposing design decisions to elementary details.***
  - ***Isolating design aspects that are not truly interdependent.***
  - ***Delaying decisions* about specific details until they are absolutely needed.**
  - ***Carefully demonstrating that each successive step in the refinement process is a faithful expansion of previous steps.***


Problem
Step 1
Step 2
Step 3
1.1
1.2
2.1
2.2
3.1
3.2
1.1.1
1.1.2
1.2.1
1.2.2
1.2.3
2.1
2.2.1
2.2.2
3.1.1
3.1.2
3.2.1
3.2.2

Modularity Cohesion coupling

Modularity

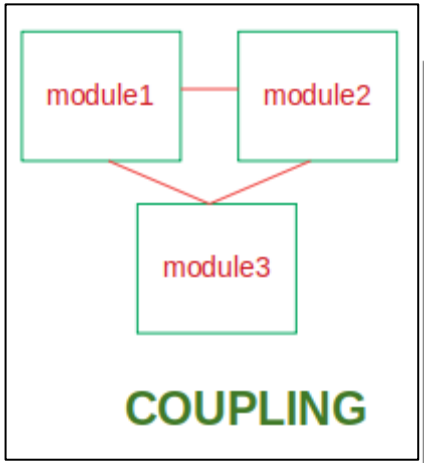
- The system is ***decomposed into a number of modules and the relationships between those modules are indicated***.
- Modularity makes the components of the ***software reusable***.
- Modularity splits a large program into smaller parts, making it easier to develop, test, and manage.

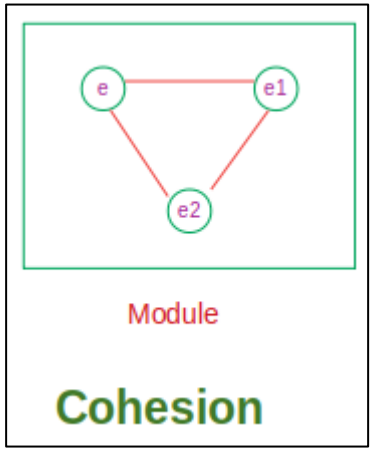


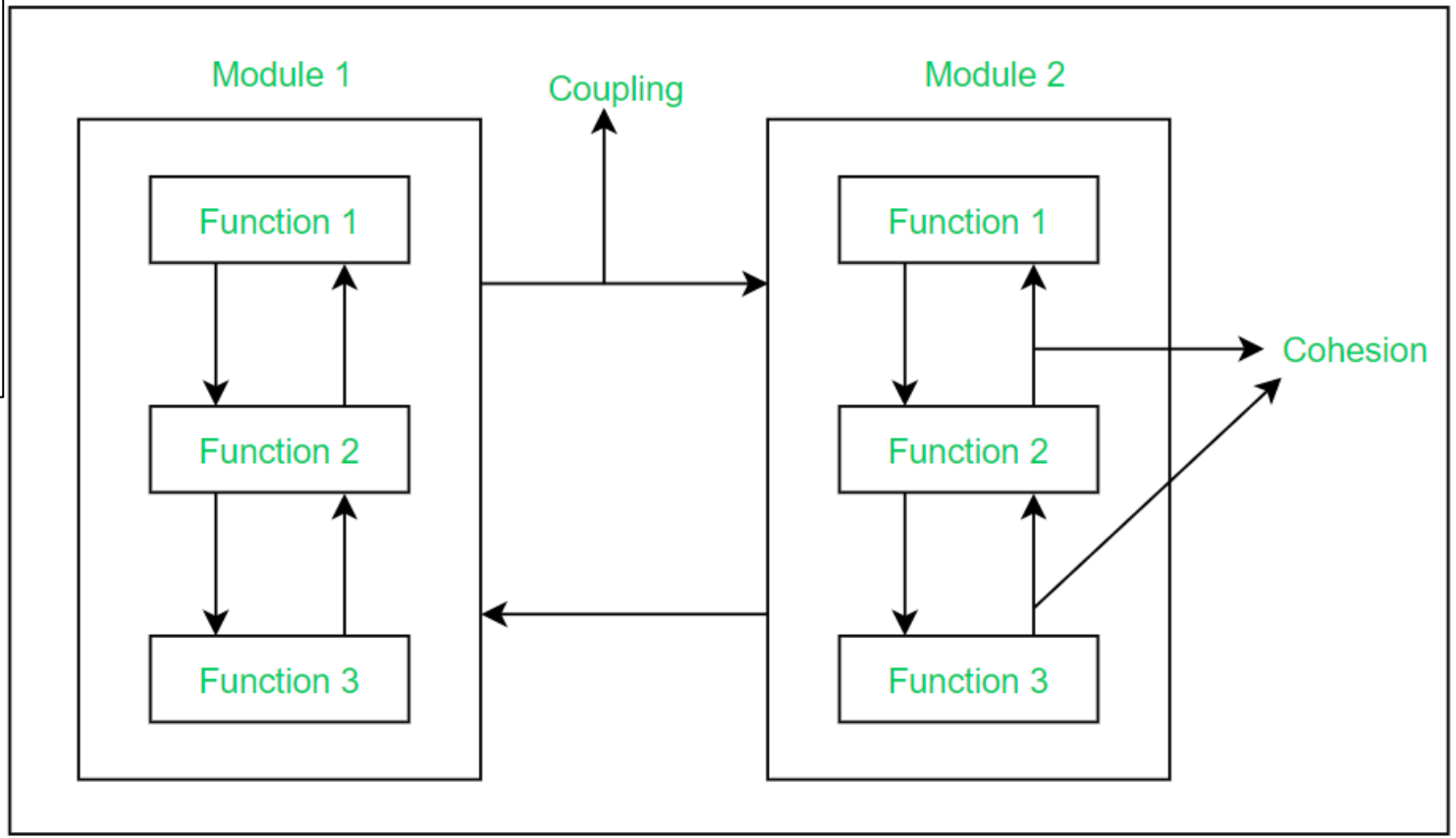
- 5 criteria to evaluate a design method with respect to its ability to define effective modular system

Modular Decomposability	Provides a systematic approach for decomposing the problem into subproblems
Modular Composability	Enables existing(reusable) design components to be assembled into a new system
Modular Understandability	Module can be understood as a standalone unit (no need to refer to other modules)
Modular Continuity	Small changes to the system requirements result in changes to individual modules
Modular Protection	Unexpected condition occurs within a module & its effects are constrained within that module

- It is measured using two criteria:
- **Cohesion:** It measures the ***relative function strength of a module***.
- **Coupling:** It measures the ***relative interdependence among modules***.

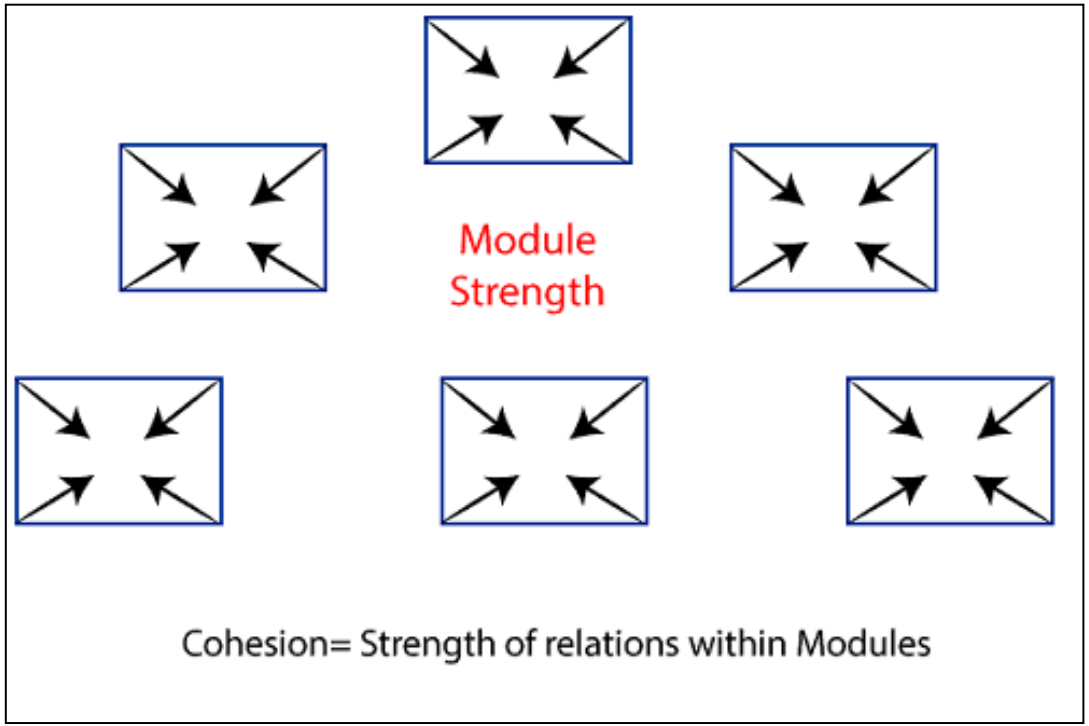




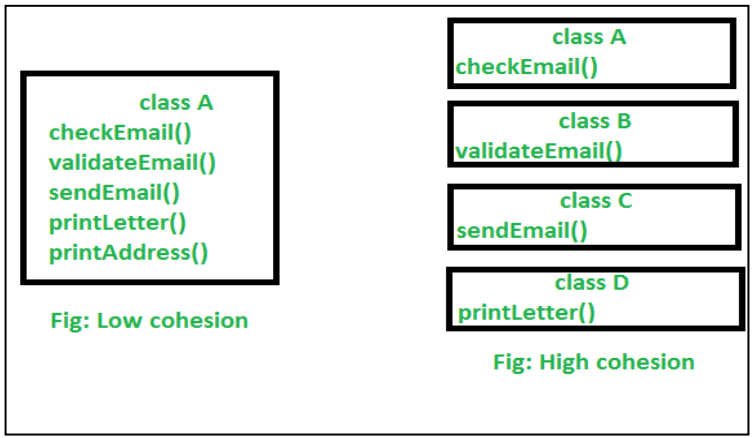


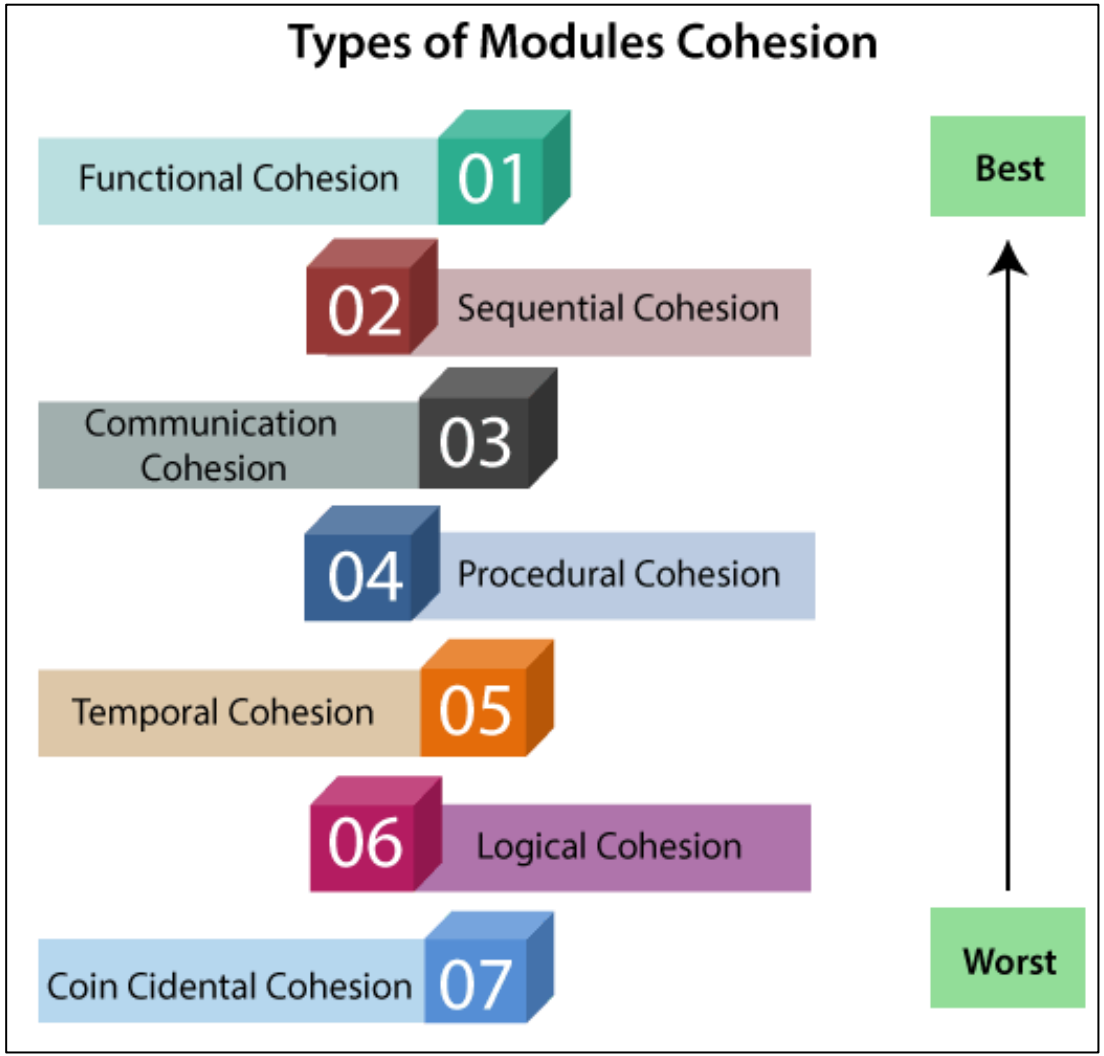
A. COHESION

- Cohesion may be viewed as the glue that keeps the module together.
- It is a measure of the mutual affinity of the elements of a module.
- Cohesion measures the strength of relationships between pieces of functionality within a given module.
- A good software design will have high cohesion.



- High (Best) cohesion indicates that the elements within a module are closely related and contribute collectively to a specific functionality.
- Low (Worst) cohesion suggests that the elements are less focused and may serve multiple unrelated purposes.





1. Functional Cohesion

- Functional Cohesion is said to exist if the ***different elements of a module, cooperate to achieve a single function***.
- All the elements within the module contribute to achieving the same objective.

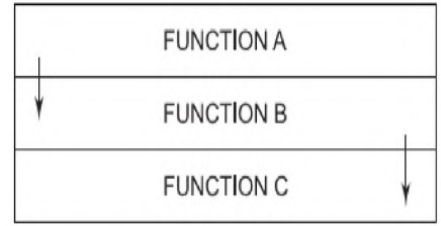
FUNCTION A Part 1
FUNCTION A Part 2
FUNCTION A Part 3

- This type of cohesion is considered the ***most desirable & strongest***.
- Examples :
  - Reading transaction records, Seat assignment to an airline passenger

```
#include <stdio.h>
// Function A Part 1: Addition
int add(int a, int b)
{
return a + b;
}
//Function A Part 2: Subtraction
int subtract(int a, int b)
{ return a - b; }
//Function A Part 3: Multiplication
int multiply(int a, int b)
{ return a * b; }
```

```
int main()
{
int num1, num2;
printf("Enter two numbers: ");
scanf("%d %d", &num1, &num2);
printf("Addition: %d\n", add(num1, num2));
printf("Subtract: %d\n", subtract(num1, num2));
printf("Multiply: %d\n", multiply(num1, num2));
return 0;}
```

2. Sequential Cohesion:



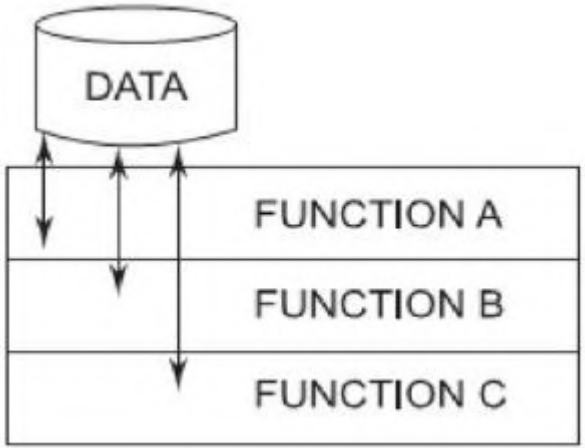
- Elements are organized in a linear sequence, where the ***output of one element becomes the input of the next***.
- The elements are executed in a step-by-step manner to achieve a particular functionality.
- Examples:
  - Cross-validated records & formatting of a module, raw records usage

```
#include <stdio.h>
// Function A
int add(int a, int b)
{
return a + b;  }
//Function B
int multiply (int sum, int c)
{
return sum * c; }
//Function A
int subtract (int product, int b)
{
return product - d; }
```

```
int main()
{
int num1, num2,num 3, num 4;
printf("Enter four numbers: ");
scanf("%d %d %d %d",&num1,&num2,&num3, &num4);
int sum = add(num1, num2);
int product = multiply(sum, num3);
int result = subtract(product, num4);
printf("Final result: %d\n", result);
return 0;}
```

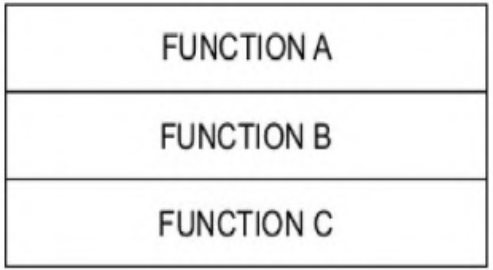
3. Communicational Cohesion

- Communicational cohesion ***occurs when the elements within a module operate on the same input data or share data through parameters.***
- The elements within the module work together by passing data to each other.
- Example :
  - When a user requests to withdraw money from an ATM, multiple elements within the module work on the same input data (account details, requested amount) and share data through parameters.
  - The set of functions defined on an array or a stack.



4. Procedural Cohesion

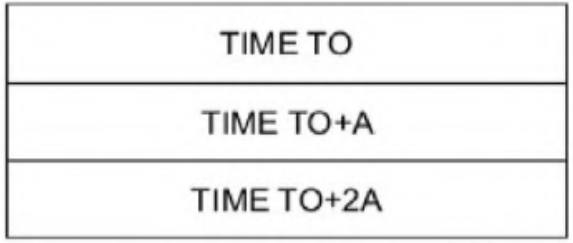
- Procedural cohesion ***occurs when the elements within a module are grouped based on a specific sequence of actions or steps.***




```
def process_order(order_id):
    validate_order(order_id)
    calculate_total(order_id)
    generate_invoice(order_id)
    send_confirmation(order_id)

def validate_order(order_id):
    print(f"Validating order {order_id}")

def calculate_total(order_id):
        print(f"Calculating total for order {order_id}")

def generate_invoice(order_id):
        print(f"Generating invoice for order {order_id}")

def send_confirmation(order_id):
        print(f"Sending confirmation for order {order_id}")

process_order(101)
```

5. Temporal Cohesion

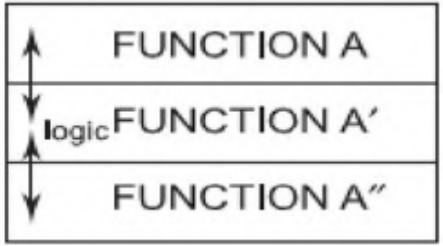
- Temporal cohesion occurs when the elements within a ***module are executed at the same time or within the same timeframe.***



- The elements are related by their timing involved.
- Example
  - Login and shutdown processes

TIME TO → Represents the initial stage (e.g., system startup).
TIME TO + A → Represents the next stage (e.g., user login operations).
TIME TO + 2A → Represents the final stage (e.g., system shutdown or session termination).

6. Logical Cohesion



- Logical cohesion occurs when the elements within a module are logically related, but do not fit into any other cohesion types.
- Elements are ***logically related and not functionally***.
- Example :

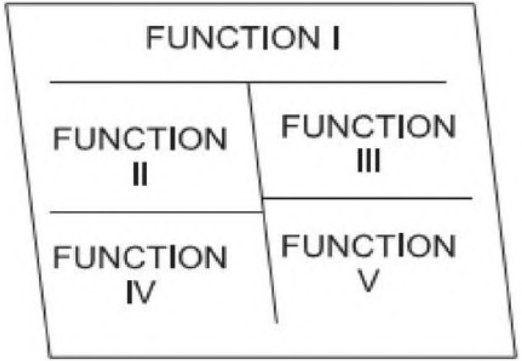
✓ FUNCTION A might validate user input for numbers
✓ FUNCTION A' for email addresses
✓ FUNCTION A" for phone numbers.
All are input validations, but distinct ones.

✓ FUNCTION A generates a summary report
✓ FUNCTION A' generates a detailed report
✓ FUNCTION A" generates a graphical report.
All create reports, but of different types.

Imagine a toolbox labeled "Tools." Inside, you find a hammer, a screwdriver, and a wrench. They are all "tools" (logically related), but using one doesn't directly involve using the others. They are independent.

7. Coincidental Cohesion:

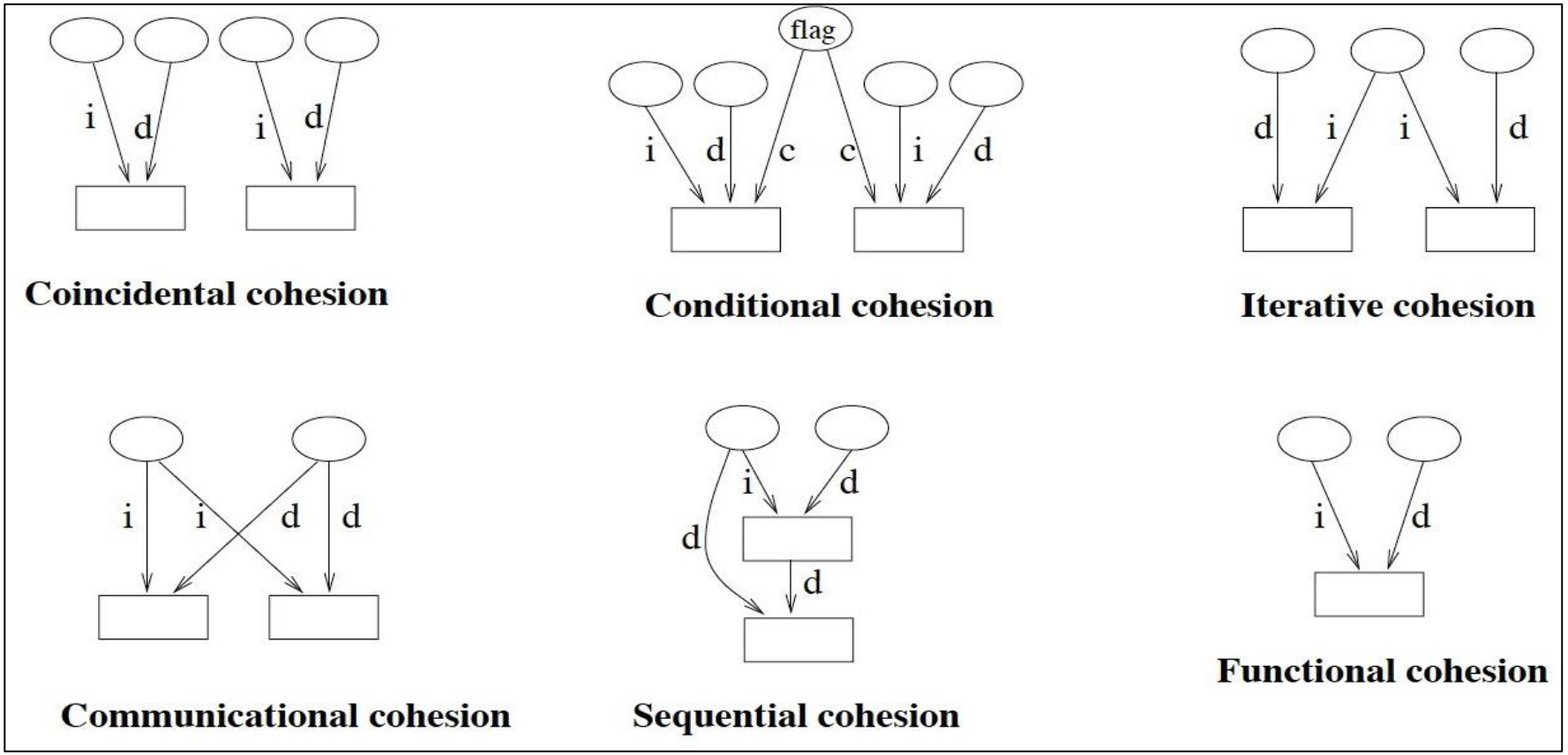
- Coincidental cohesion occurs when the elements are not related to each other.
- Example
  - print next line and reverse the characters of a string in a single component.



FUNCTION I: Opens the file.
FUNCTION II: Reads data from the file.
FUNCTION III: Writes data to the file.
FUNCTION IV: Closes the file.
FUNCTION V: Deletes the file.

Think of a remote control for a TV. It has buttons for:

- Power on/off
- Volume up
- Volume down
- Channel up
- Channel down


i
d
i
d
Coincidental cohesion
flag
i
d
c
c
i
d
Conditional cohesion
d
i
i
d
Iterative cohesion
i
i
d
d
Communicational cohesion
i
d
d
d
Sequential cohesion
i
d
Functional cohesion

COUPLING

- Coupling in software engineering refers to the ***degree of interdependence*** & ***connection*** between modules or components within a software system.
- Two modules are said to have high coupling if they are closely connected.
- The resource that can be identified are
  - functional behavior
  - a common interface
  - a common field in a user interface
  - the data picked from a field and shared between two modules
  - a transaction shared between two modules

- Coupling is measured by the number of relations between the modules.
- That is, the coupling increases as the number of calls between modules increase or the amount of shared data is large.

Functional behavior

- A login module and an authentication module work together; the login module calls the authentication module to verify user credentials.

A common interface:

- Different payment methods (credit card, PayPal) use the same payment gateway interface to process transactions.

A common field in a user interface :

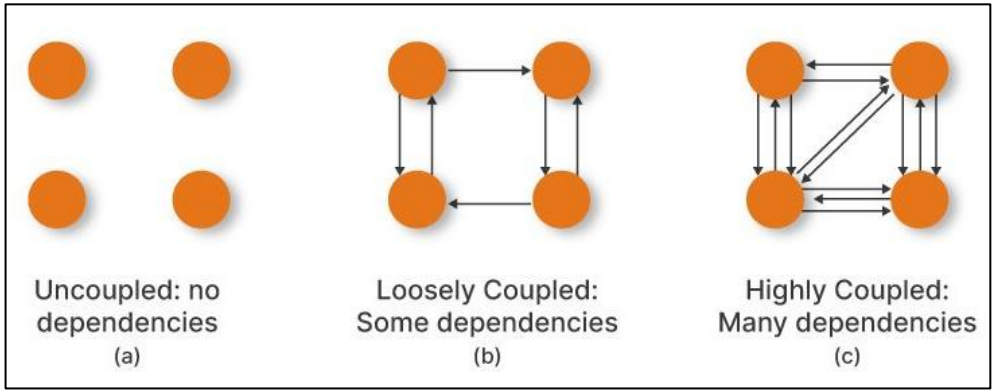
- A "Customer ID" field in an order form is also used in the customer details section.

The data picked from a field and shared between two modules

- A user's email entered in a signup form is also used in the newsletter subscription module.

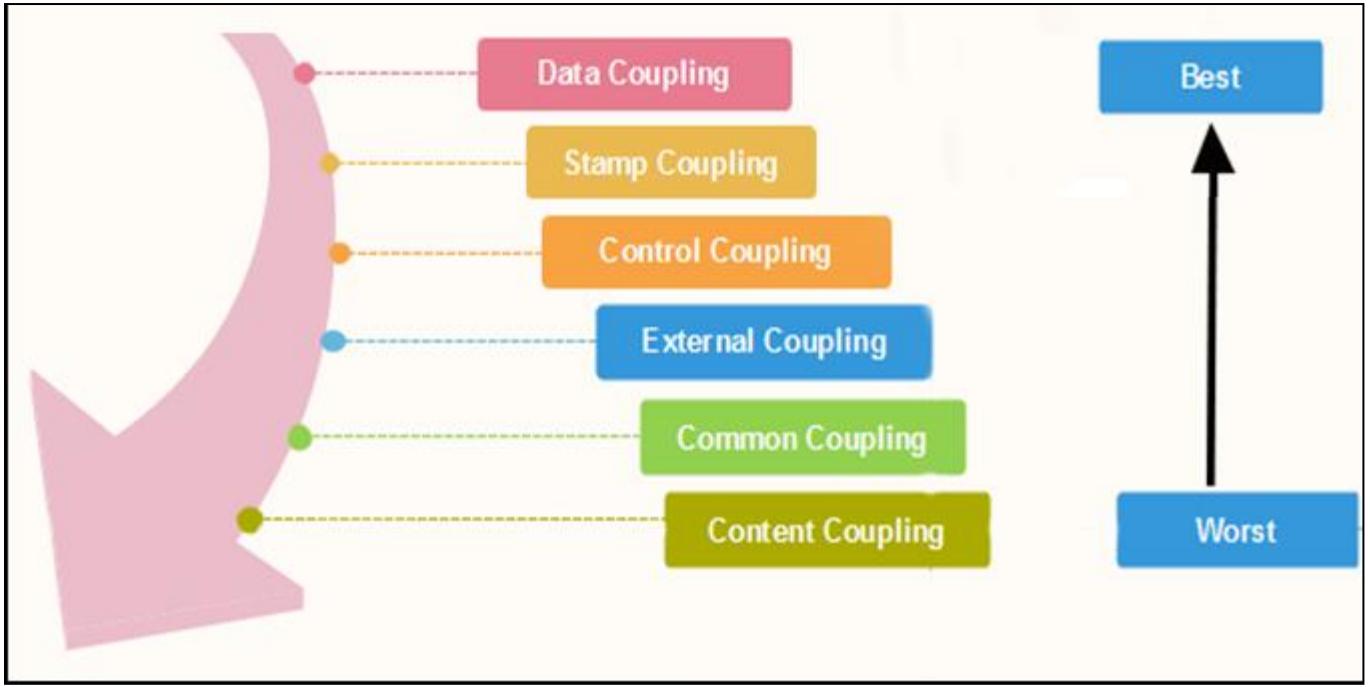
A transaction shared between two modules:

- An e-commerce checkout process where the order module updates inventory and triggers the payment module.

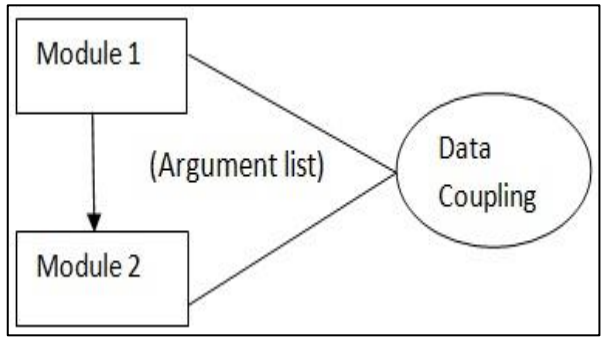


- High coupling generates more errors because they share large number of data

Types of coupling (DSCECC)



1. Data Coupling:



- Data coupling occurs when ***modules share data through parameters or arguments.***
- Each module maintains its own data and does not exactly access or modify the data of other modules.
- This type of coupling promotes ***encapsulation*** & ***module interdependence***.

Consider a banking system with two modules:

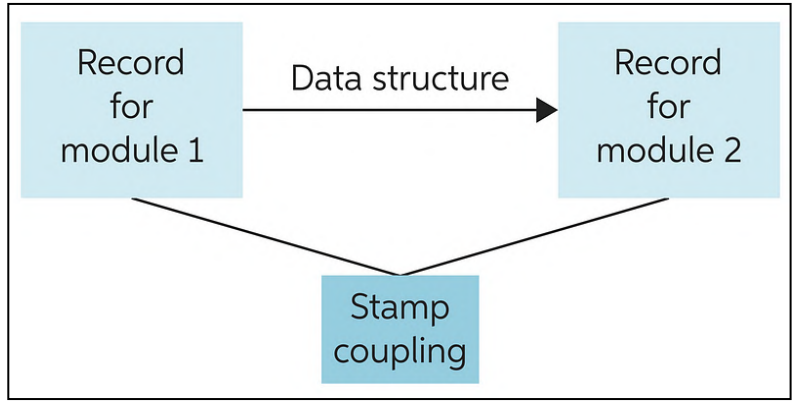
- **Account Module** – Handles account details.
- **Transaction Module** – Processes deposits and withdrawals.

```
void update_balance(int accountNumber, float
                                        amount)
{
 printf("Updating account %d \n",
                        accountNumber, amount);
}

void deposit(int accountNumber, float amount)
 {
 update_balance(accountNumber, amount);
}
```

```
int main() {
int accNumber = 12345;
float depositAmount = 1000.50;
deposit(accNumber,depositAmount);
return 0; }
```

2. Stamp Coupling

- Stamp coupling happens when modules share data by passing ***entire data structures*** (like structs in C) instead of just the needed values.
- Some fields in the structure may not be used by the receiving module.
- No junk or unused data will be shared between the two coupling modules.




```
struct Student
{
int id;
char name[50];
float marks;
};
```

```
void generateReport(struct Student s)
{
 printf("Generating report for marks: %f\n", s.marks);

int main()
{
struct Student student1 = {101, "Alice", 85.5};
generateReport(student1);
return 0;}
```

3. Control Coupling:

- Control coupling happens when **one module controls another module's behavior** by passing control data like **flags** **or** **switches**.
- One module knows too much about another's internal logic.
- If you change one module, the other must also be changed.
- This makes the system harder to maintain.

A smart home system has a central controller module that determines whether the air conditioner (AC) module should operate in cooling or heating mode based on a temperature sensor's reading. The controller passes a flag to the AC module, influencing its internal logic.

5. External Coupling:

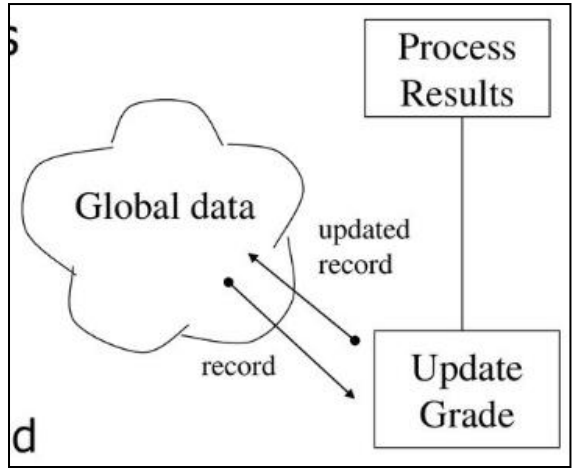
- External coupling occurs when ***two modules depend on the same external factors***.
- Examples include:
  - Both modules receive data from the same legacy application.
  - Both require the same hardware component.
  - Both use a shared file or folder.
  - Both communicate through the same network switch or router.

Imagine a **patient monitoring system** and a **hospital billing system** in a hospital.

- Both systems receive patient details from the same **electronic health record (EHR) system** (common external data source).
- Both require the same **biometric device** (e.g., a heart rate monitor) for real-time data.
- Both access a shared **patient database** to update and retrieve records.
- Both communicate through the same **hospital network router** (network dependency).

6. Common Coupling:

- Common coupling occurs when two or more modules in the system ***share global data***.
- The modules can access & manipulate the same global variables & the data structures.



In a hospital management system, multiple modules share global patient records stored in a central database.

- **Doctor's Module:** Updates the patient's diagnosis and treatment plan.
- **Pharmacy Module:** Accesses the same global data to check prescribed medications.
- **Billing Module:** Calculates the total bill based on treatments and medications.

If the Doctor's Module updates a patient's prescription incorrectly, the Pharmacy Module might dispense the wrong medicine.

6. Content Coupling

- Content coupling occurs when one module ***directly accesses or modifies*** the content of another module.
- Content coupling should never be used when designing modules..
- One module depends on the implementation of the other module.

Imagine a simple game with two modules:
Player Module: Manages player information, including the player's name and score.
Score Display Module: Responsible for displaying the player's score on the screen.

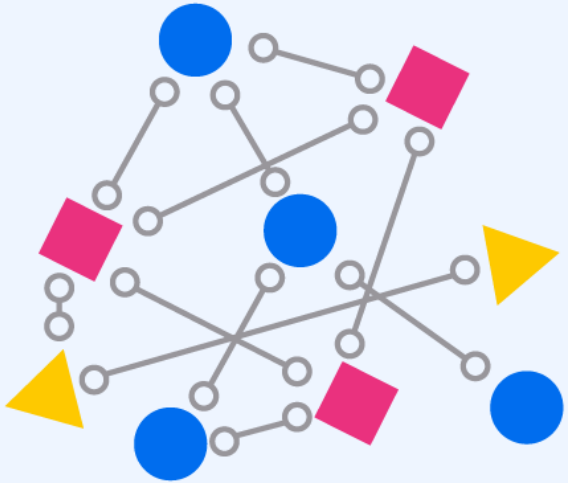
Direct Access:

- The Score Display Module directly reaches into the Player Module's internal memory or data structure where the score is stored. It knows the exact memory address or variable name where the score is kept.

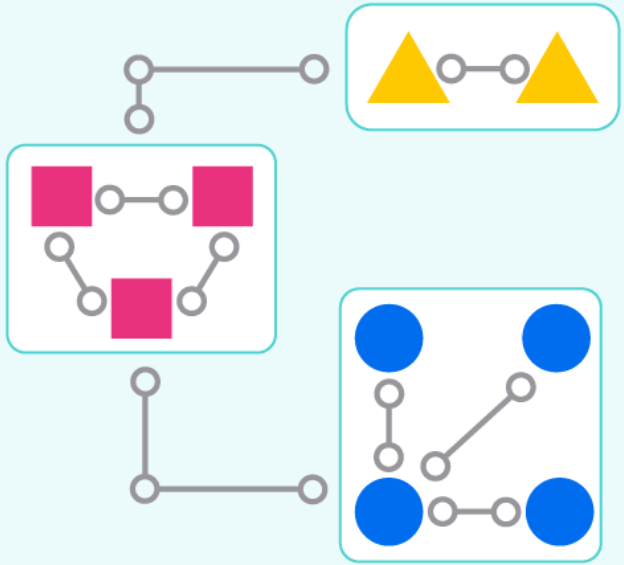
Modification:

- Even worse, the Score Display Module might directly change the player's score in the Player Module's internal data, without going through any proper "update score" function in the Player Module.

Cohesion + Coupling



Without



With

Cohesion vs Coupling

vs

vs

ASPECT	COHESION	COUPLING
Definition	Cohesion refers to how closely related and focused the responsibilities of a single module or component are. High cohesion means that a module performs a set of related tasks.	Coupling refers to the degree of interdependence between software modules. It indicates how much a module relies on other modules.
Objective	To ensure that a module or component has a clear and singular purpose, making it easier to understand, maintain, and reuse.	To manage the dependencies between modules, aiming for loose coupling to enhance flexibility, reusability, and maintainability.
Impact	High cohesion generally leads to better maintainability, readability, and reusability of code.	Low coupling helps in making the system more flexible, easier to test, and easier to maintain.
Examples	A module that calculates payroll might have methods for calculating gross pay, taxes, and net pay—all related to payroll processing.	Tight coupling occurs when a module directly references variables or methods of another module. For example, Module A calling functions from Module B directly without using interfaces.

ARCHITECTURAL DESIGN

- Architectural design is concerned with ***understanding how a software system should be organized and designing the overall structure of that system***.
- It is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them.
- The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.
- Software architectures can be designed at two levels of abstraction

1. *Architecture in the small*

- Concerned with the architecture of ***individual programs***.
- At this level, we are concerned with the way that an ***individual program is decomposed into components***.
- This level of architecture ensures that the program is well-structured and efficient.

Consider a to-do list application built using Python and Flask.
The application consists of:

- **Frontend** (HTML, CSS, JavaScript) for user interaction.
- **Backend** (Flask) to handle user requests.
- **Database** (SQLite) to store tasks.

2. *Architecture in the large*

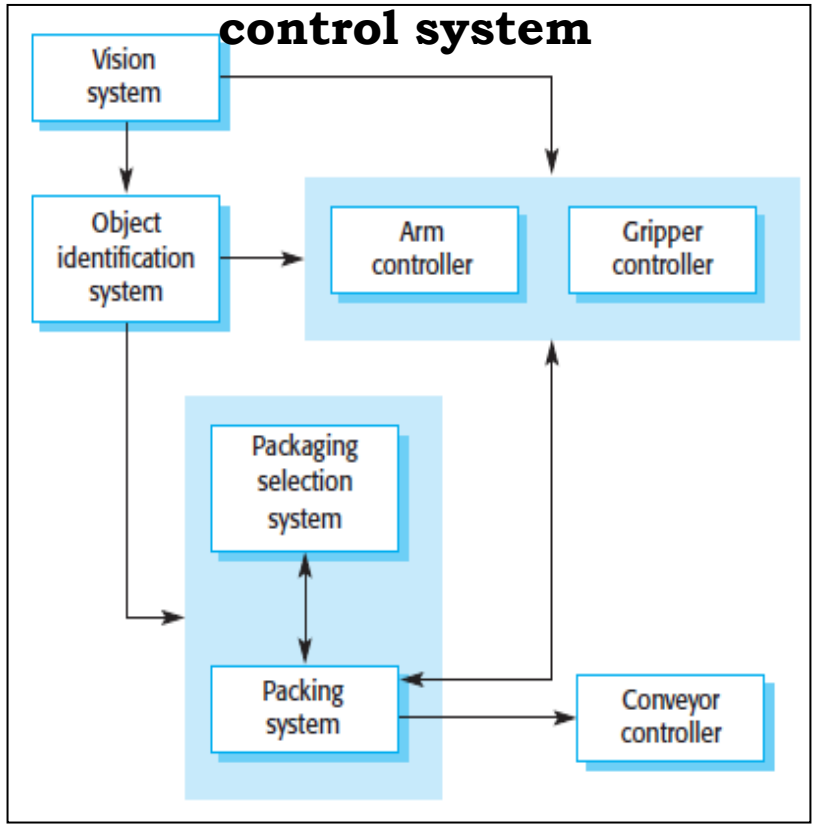
- Concerned with the architecture of ***complex enterprise systems*** that include **other systems**, **programs**, and **program components**.
- These enterprise systems may be ***distributed over different computers***, which may be owned and managed by different companies

A Library Management System (LMS) in a university may have:

- A **web application** for students to search and borrow books.
- A **database** for storing book details and user records.
- A **payment integration** for overdue fines.
- A **third-party book supplier system** to request new books.
- A **cloud storage** for e-books and digital materials.

- System architectures are often modeled informally using ***simple block diagrams***.
- This **robotic system** can pack different kinds of objects.
- It uses a vision component to
  - ***Pick out objects on a conveyor***
  - ***Identify the type of object***
  - ***Select the right kind of packaging***.
- The system then moves objects from the delivery conveyor to be packaged.
- It places packaged objects on another conveyor.
- The architectural model shows these components and the links between them.

Architecture of a packing robot control system



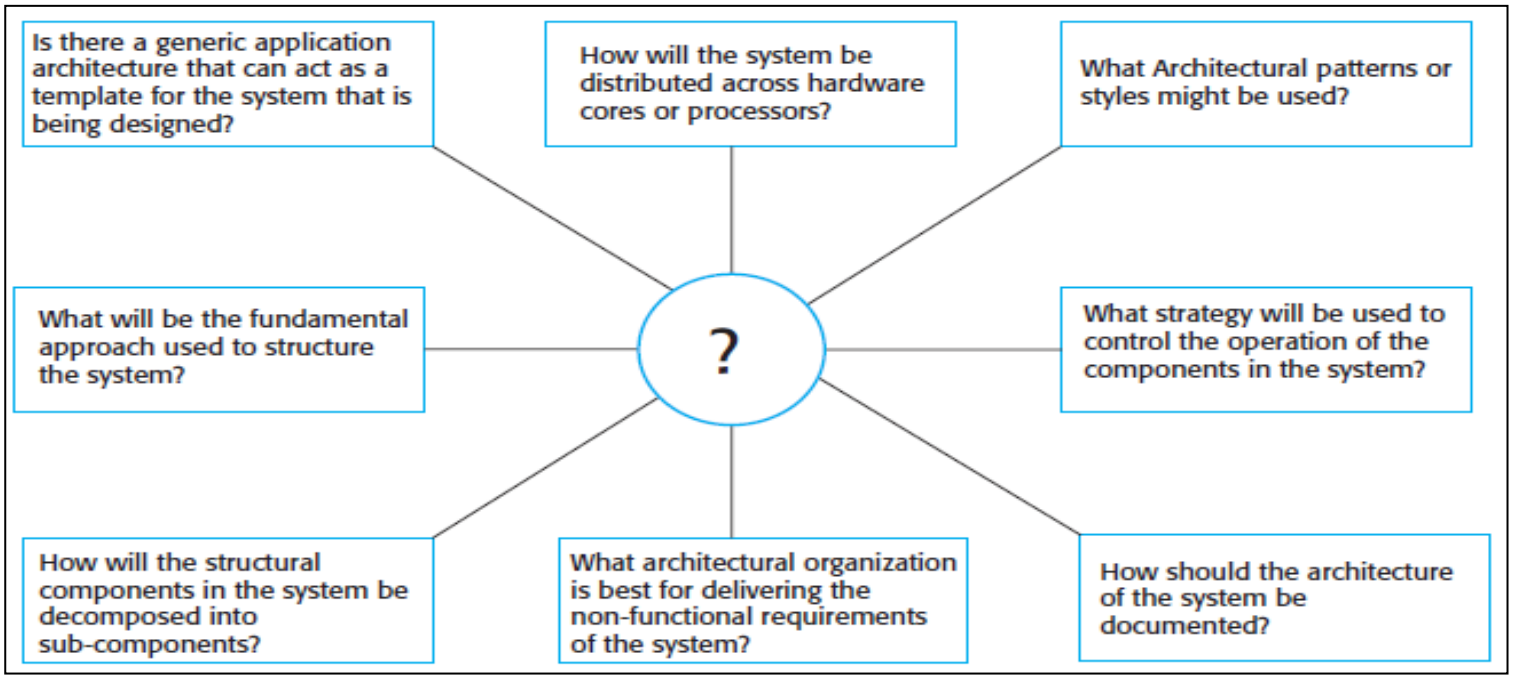
- Each **box** in the diagram ***represents a component***.
- **Boxes within boxes** indicate that the ***component has been decomposed to subcomponents***.
- **Arrows** mean that ***data and or control signals are passed from component to component*** in the direction of the arrows.

- Bass et al. suggest that explicitly designing and documenting software architecture has three advantages:

1. ***Stakeholder communication***
   - Architecture is a **high-level design of a system** that shows how different parts work together.
   - It helps different people (developers, managers, users, and business owners) understand the system without getting into too many technical details.
2. ***System analysis***
   - Making the system architecture explicit at an early stage in the system development requires some analysis.
   - Architectural design decisions have a deep effect on whether or not the system can meet critical requirements such as ***performance***, ***reliability***, and ***maintainability***.
3. ***Large-scale reuse***
   - An architectural model is a compact, manageable description of how a system is organized and how the components interoperate.
   - The system architecture is often the same for systems with similar requirements and so can support large-scale software reuse

1. Architectural Design Decisions

- Architectural design is a creative process in which you design a ***system organization that will satisfy the functional and non-functional requirements of a system***.
- It depends on the
  - ***type of system being developed***
  - ***Background and experience of the system architect***
  - ***Specific requirements for the system.***



- The choice of architectural style and structure should depend on the **non-functional requirements** of the system:

(i) Performance :

- For better performance, keep critical operations within a few components and run them on the same computer instead of spreading them across a network.

(ii) Security

- If security is a critical requirement, a ***layered structure*** for the architecture should be used, with the most ***critical assets protected in the innermost layers*** and a high level of security validation applied to these layers.

(iii) Safety

- If safety is a critical requirement, the architecture should be designed so that safety-related operations are ***co-located in a single component or in a small number of components.***
- This reduces the costs and problems of safety validation and may make it possible to provide related protection systems that can safely shut down the system in the event of failure.

(iv) Availability

- If availability is a critical requirement, the architecture should be ***designed to include redundant components*** so that it is possible to replace and update components without stopping the system.

(v) Maintainability

- If maintainability is a critical requirement, the system architecture should be designed using fine-grain, self-contained components that may readily be changed.
- ***Producers of data should be separated from consumers, and shared data structures should be avoided.***

2. Architectural views

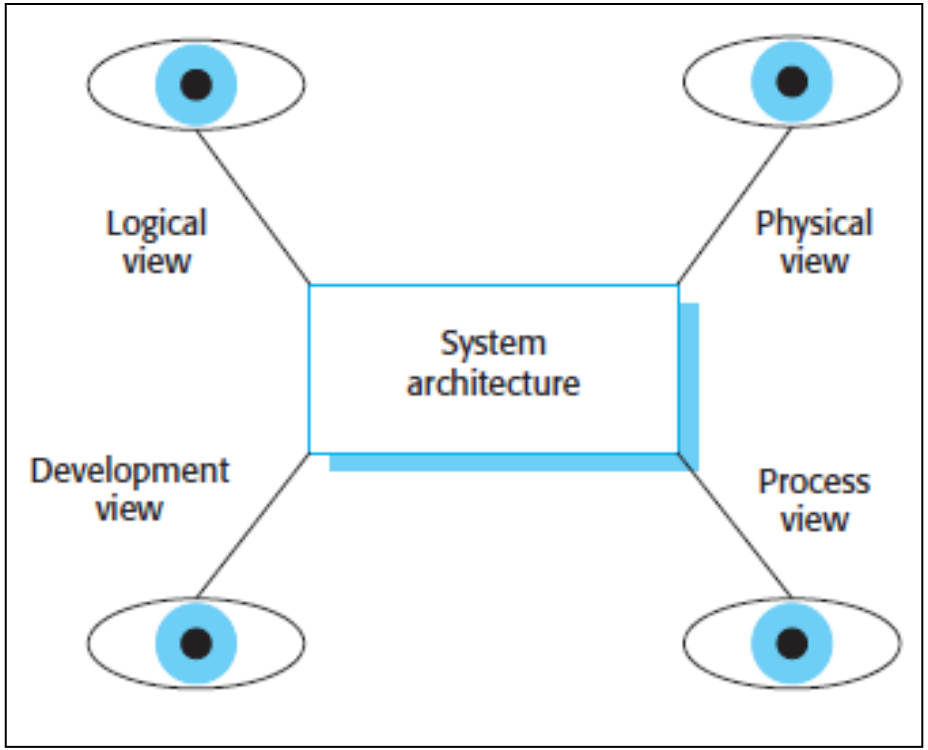
- There should be four fundamental architectural views, which can be linked through common use cases or scenarios

1. *A logical view*

- Shows the ***key abstractions in the system as objects or object classes***.
- It should be possible to relate the system requirements to entities in this logical view.

2. *A process view*

- Shows ***how, at runtime, the system is composed of interacting processes.***
- This view is useful for making judgments about non-functional system characteristics such as performance and availability.



3. *A development view*

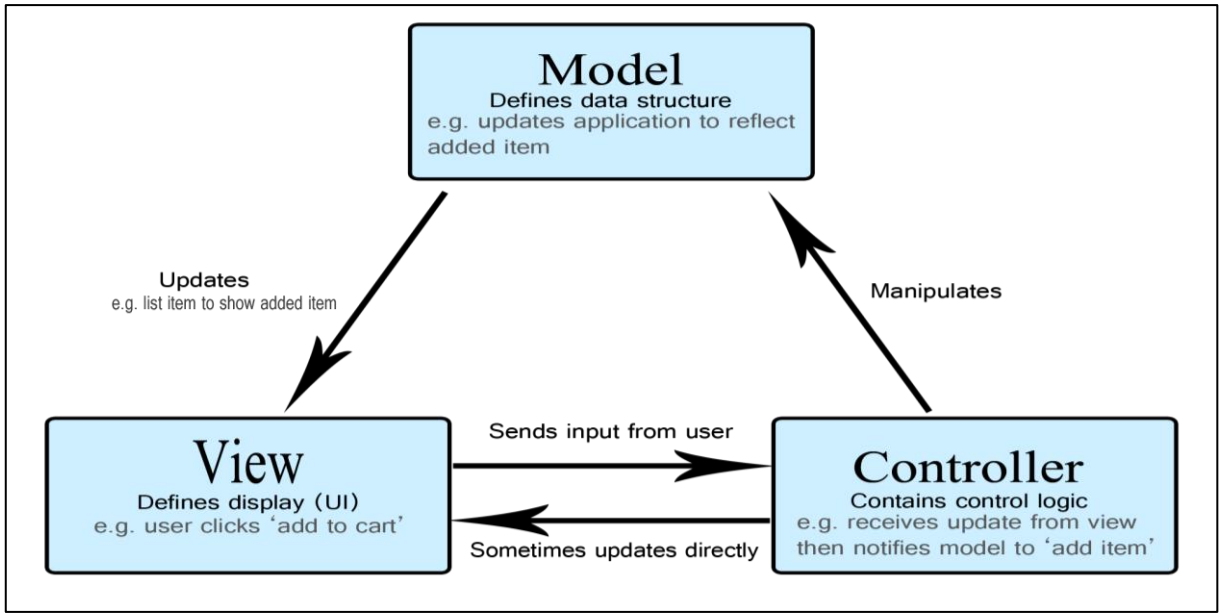
- Shows ***how the software is decomposed for development***; that is, it shows the breakdown of the software into components that are implemented by a single developer or development team.
- This view is useful for software managers and programmers.

4. *A physical view*

- Shows the ***system hardware and how software components are distributed across the processors in the system.***
- This view is useful for systems engineers planning a system deployment

3. Architectural Patterns

- Patterns may be described in a standard way using ***a mixture of narrative description and diagrams***.
- The **stylized pattern description** includes the ***pattern name***, a ***brief description***, a ***graphical model***, and an ***example*** of the type of system where the pattern is used.
- **Model-View-Controller pattern** is the basis of interaction management in many **web-based systems** and is supported by most language frameworks.



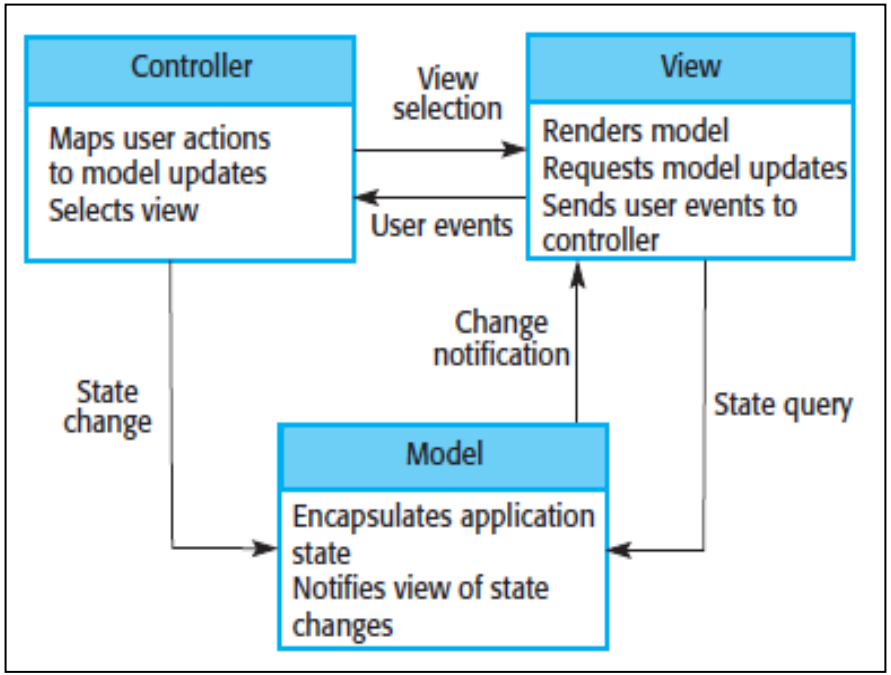
Name	Name MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. • **Model** component ***:*** Manages system data and logic. • **View** component : Defines and manages how data is presented to the user. • **Controller** component ***manages user interaction*** (e.g., key presses, mouse clicks, etc.) ***and passes these interactions to the View and the Model***.
Example	Architecture of a web-based application system organized using the MVC pattern is shown in the next slide.
When used	• Used when there are multiple ways to view and interact with data. • Used when the future requirements for interaction and presentation of data are unknown.
Advantages	• Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways, with changes made in one representation shown in all of them. (Spreadsheet and chart application)
Disadvantages	May involve additional code and code complexity when the data model and interactions are simple.

In a restaurant, the Chef is the Model. It's job is to take orders (requests) from the Waiter and create some awesome food for the Waiter to deliver to the Customer.

The Waiter is the Controller. They take in orders (requests), take them back to the Chef (Model), and then deliver the proper meal to customers (Users). The Table is the View. It just holds whatever meal the Waiter brings back from the Chef.

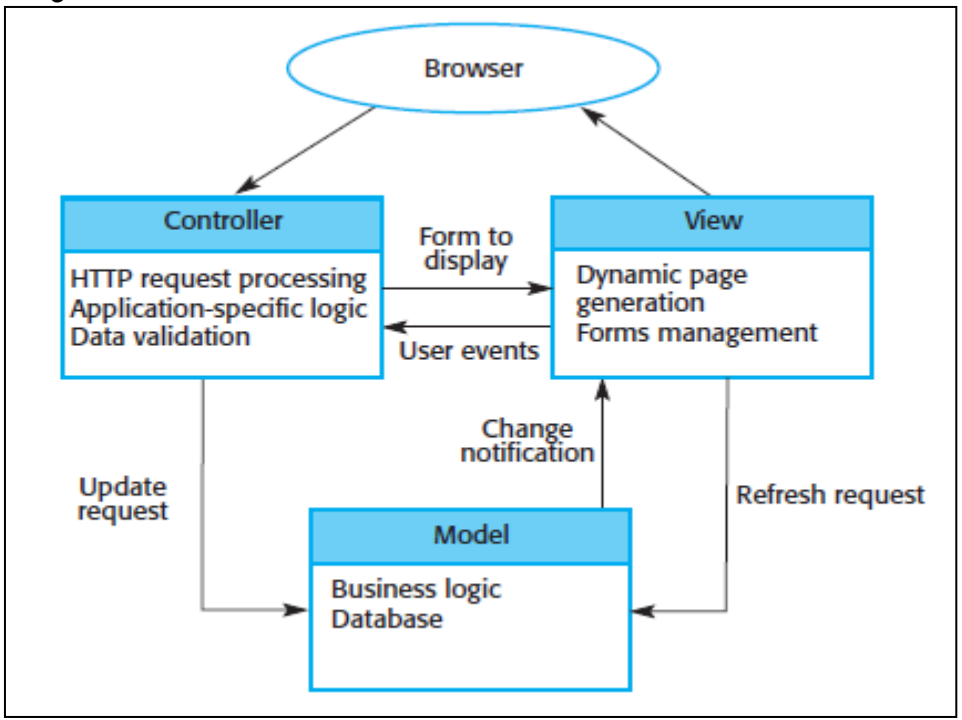
- Graphical models of the architecture associated with the MVC pattern are shown in Figures

Conceptual view



Organization of the Model-View-Controller

Runtime system architecture when this pattern is used for interaction management in a web-based system.



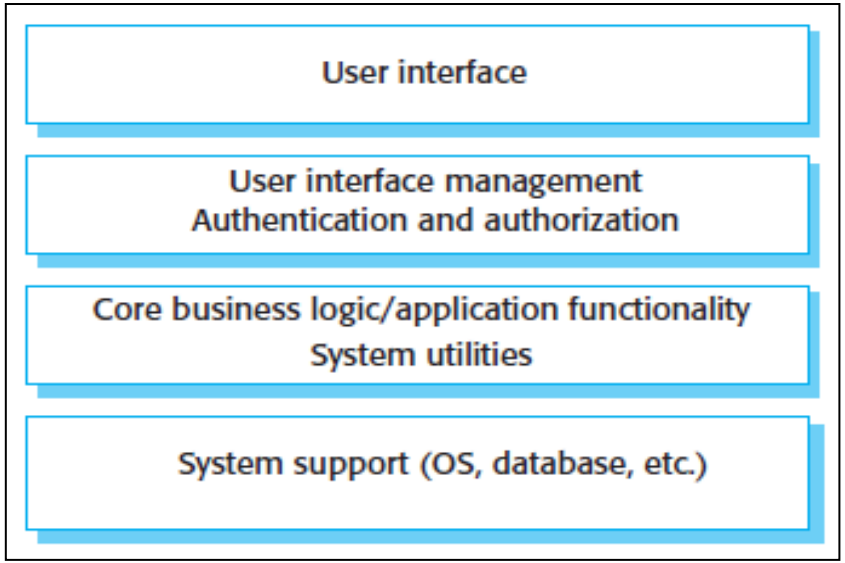
Web Database application architecture using the MVC pattern

3.1 Layered architecture

- Separation and independence in architectural design mean that different parts of a system work on their own without affecting each other.
- **Layered Architecture pattern** is another way of achieving separation and independence.
- The ***system functionality is organized into separate layers***, and each layer only relies on the facilities and services offered by the layer immediately beneath it.
- This layered approach supports the ***incremental development of systems***.
- The architecture is also changeable and portable.
- In a layered system, you can replace a layer with an improved version without affecting the rest of the system, as long as its interface remains the same.
- If a layer's interface changes or new features are added, only the next layer needs adjustments.
- This structure also helps in making applications work across different platforms by keeping machine-specific details isolated.

- The **lowest layer** includes ***system support software***—typically, database and operating system support.
- The **next layer** is the application layer, which includes the ***components concerned with the application functionality*** and ***utility components used by other application components.***
- The **third layer** is concerned with user interface management and providing user authentication and authorization
- The **top layer** providing user interface facilities.

Generic layered architecture



System Support (Bottom Layer): Kitchen (database & operating system) runs everything.

Application Layer: The chefs prepare food (application functions).

User Management Layer: The cashier checks reservations and payments (authentication & authorization).

User Interface (Top Layer): The dining area where customers order and eat (user interaction).

Name	Layered architecture
Description	Organizes the ***system into layers***, with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system.
Example	A layered model of a ***digital learning system*** to support learning of all subjects in schools is shown in next slide.
When used	Used when ***building new facilities on top of existing systems***; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multilevel security.
Advantages	Allows replacement of entire layers as long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system
Disadvantages	In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Layered Architecture pattern

A user story for the iLearn system

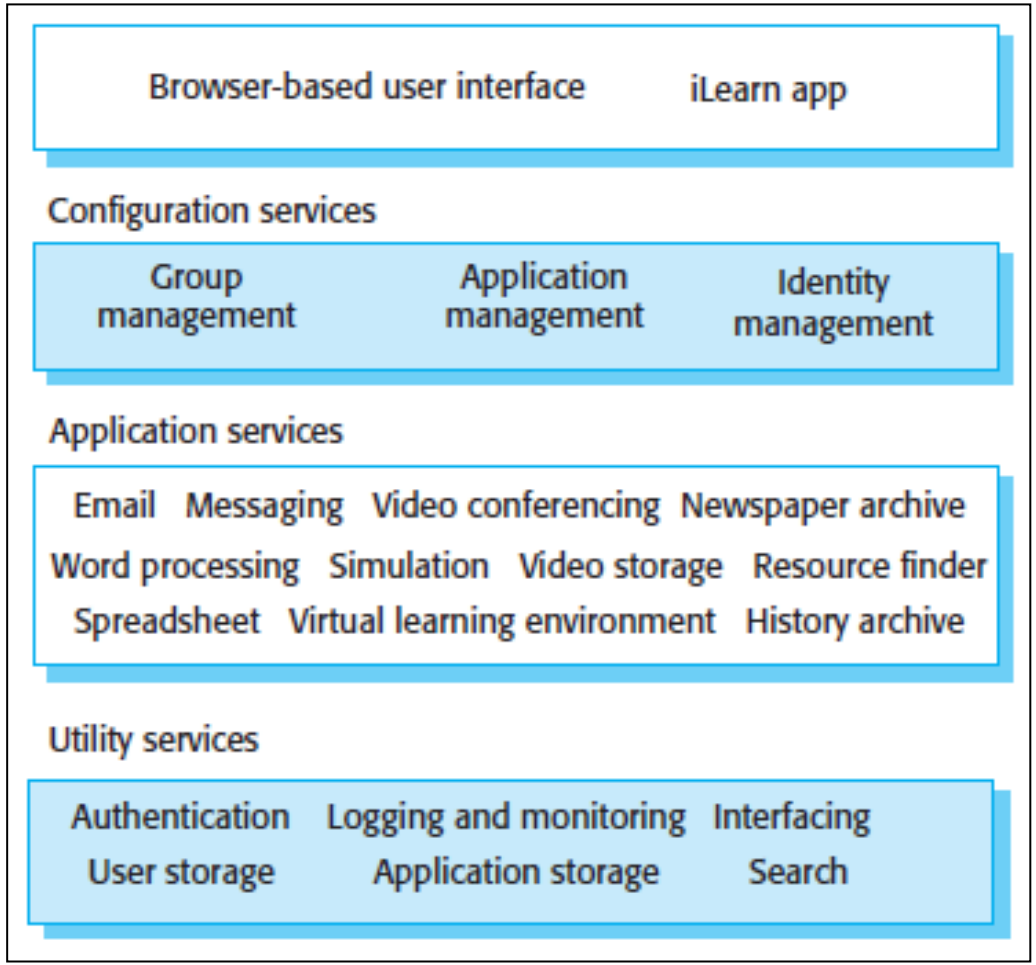
Photo sharing in the classroom

Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused on the fishing industry in the area, looking at the history, development, and economic impact of fishing. As part of this project, pupils are asked to gather and share memories from relatives, use newspaper archives, and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo-sharing site because he wants pupils to take and comment on each other's photos and to upload scans of old photographs that they may have in their families.

Jack sends an email to a primary school teachers' group, which he is a member of, to see if anyone can recommend an appropriate system. Two teachers reply, and both suggest that he use KidsTakePics, a photo-sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

This story describes a situation in a primary (elementary) school where the teacher is using the environment to support student projects on the fishing industry.

- The iLearn digital learning system has a four-layer architecture that follows this pattern



Architecture of the iLearn system

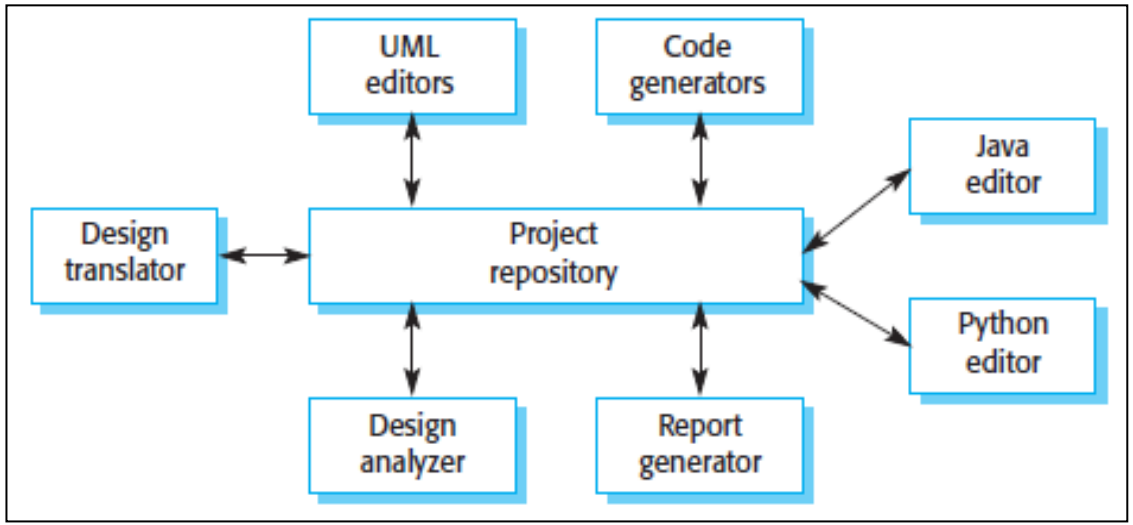
3.2 Repository Architecture

- Repository pattern describes how a ***set of interacting components can share data***.
- The majority of systems that use large amounts of data are organized around a shared database or repository.
- This model is therefore suited to applications in which data is generated by one component and used by another.
- Examples of this type of system include
  - ***Command and Control Systems***
  - ***Management Information Systems***
  - ***Computer- Aided Design (CAD) Systems***
  - ***Interactive Development Environments for Software.***

Name	Repository
Description	All data in a system is managed in a ***central repository*** that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Example of an IDE where the components use a repository of system design information is shown in next slide Each software tool generates information, which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

- The repository in this case might be a version-controlled environment that keeps track of changes to software and allows rollback to earlier versions.
- The repository is passive and control is the responsibility of the components using the repository.
- An alternative approach, which has been derived for artificial intelligence (AI) systems, uses a “blackboard” model that triggers components when particular data become available.
- This is appropriate when the data in the repository is unstructured.
- Decisions about which tool is to be activated can only be made when the data has been analyzed.

A repository architecture for an IDE



3.3 Client–Server Architecture

- Client-Server pattern is a common way to organize **distributed systems**.
- In this pattern, the system has two main parts:

  Servers – These ***provide services***, such as data storage or processing.

  Clients – These request and use the services provided by the servers.

- **Client-Server pattern** is usually used in distributed systems with separate machines, but it can also run on a single computer by keeping the client and server as separate programs.
- **Clients** may have to ***know the names of the available servers and the services they provide***.
- **Servers** do not need to know the identity of clients or how many clients are accessing their services.
- Clients use a **request-reply** system to access server services.
- They send a request (e.g., using **HTTP**) and wait for the server to respond.

Name	Repository
Description	In a client–server architecture, the system is presented as a set of services, with each ***service delivered by a separate server***. Clients are users of these services and access servers to make use of them.
Example	An example of a **film and video/DVD library** organized as a client–server system is shown in next slide
When used	The Client-Server pattern allows multiple locations to access a shared database. Servers can be copied to manage more users & workload changes easily.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	If a server fails or is attacked, the service stops working because it depends on a single server. Performance can vary because it depends on both the network speed and the server's capability. Management problems may arise if servers are owned by different organizations.

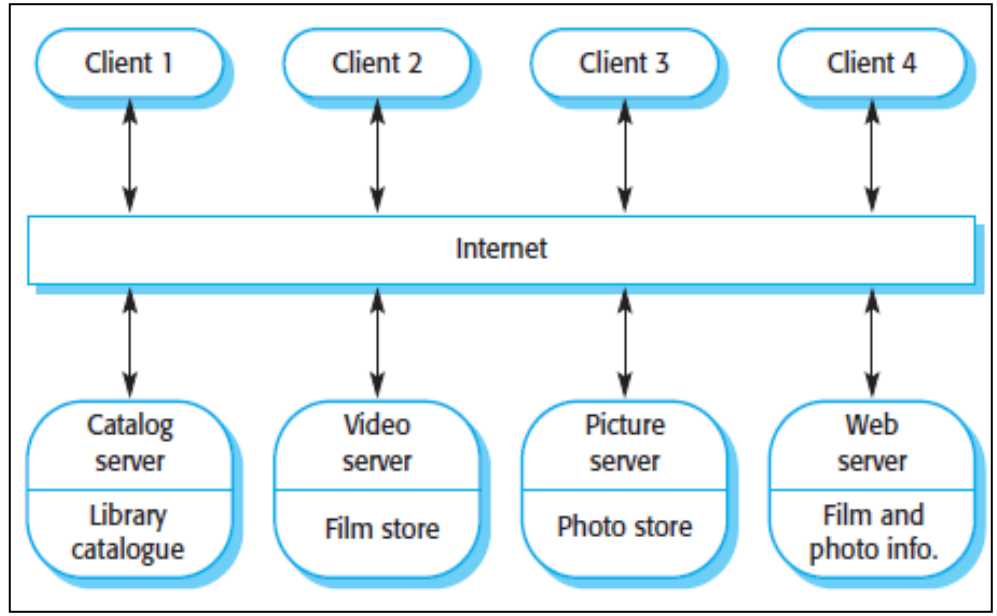
Client–Server Pattern

- The major components of this model are:

1. A **set of servers** that offer ***services to other components***.

- Examples : Include print servers that offer printing services, file servers that offer file management services, and a compile server that offers programming language compilation services.
- Servers are software components, and several servers may run on the same computer.

A client– server architecture for a film library



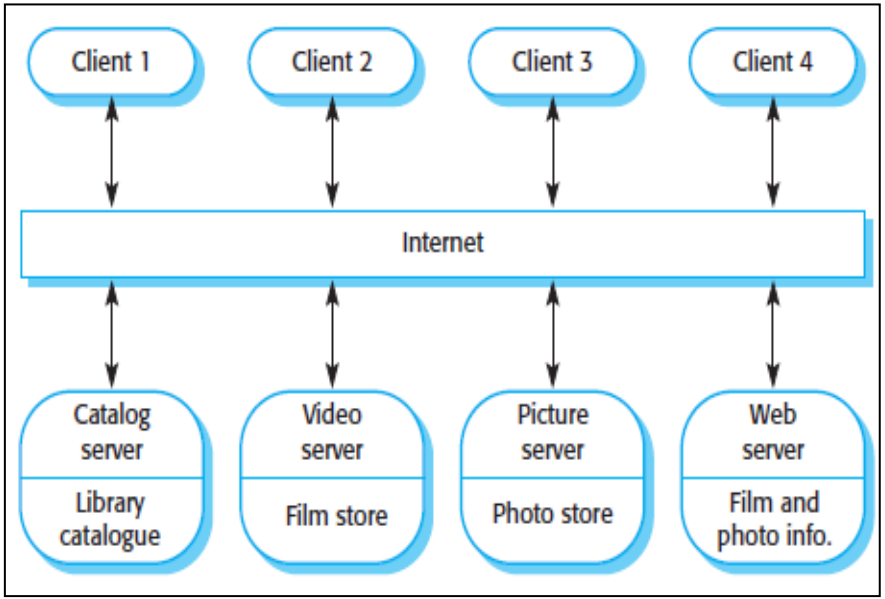
2. A **set of clients** that ***call on the services offered by servers***.

- Multiple clients can run at the same time on different computers.

3. A **network** that ***allows the clients to access these services***.

- Client–server systems are usually implemented as distributed systems, connected using Internet protocols.

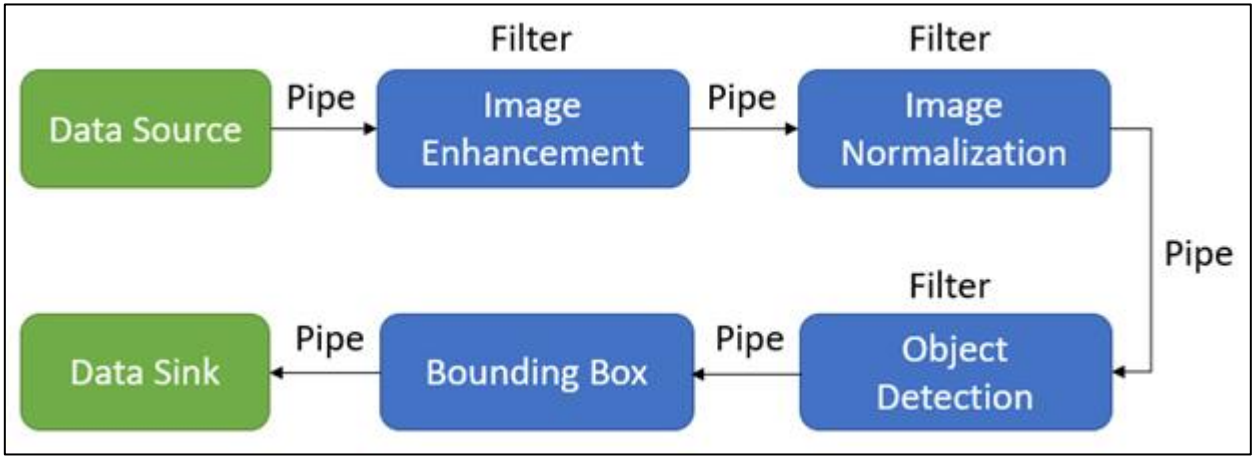
- This is a multiuser, **web-based system** for providing a film and photograph library.
- **Video frames** must be sent quickly and in sync, even at low resolution. The video server can compress and decompress videos in different formats as needed.
- **Still pictures** must be maintained at a high resolution, so it is appropriate to maintain them on a separate server.

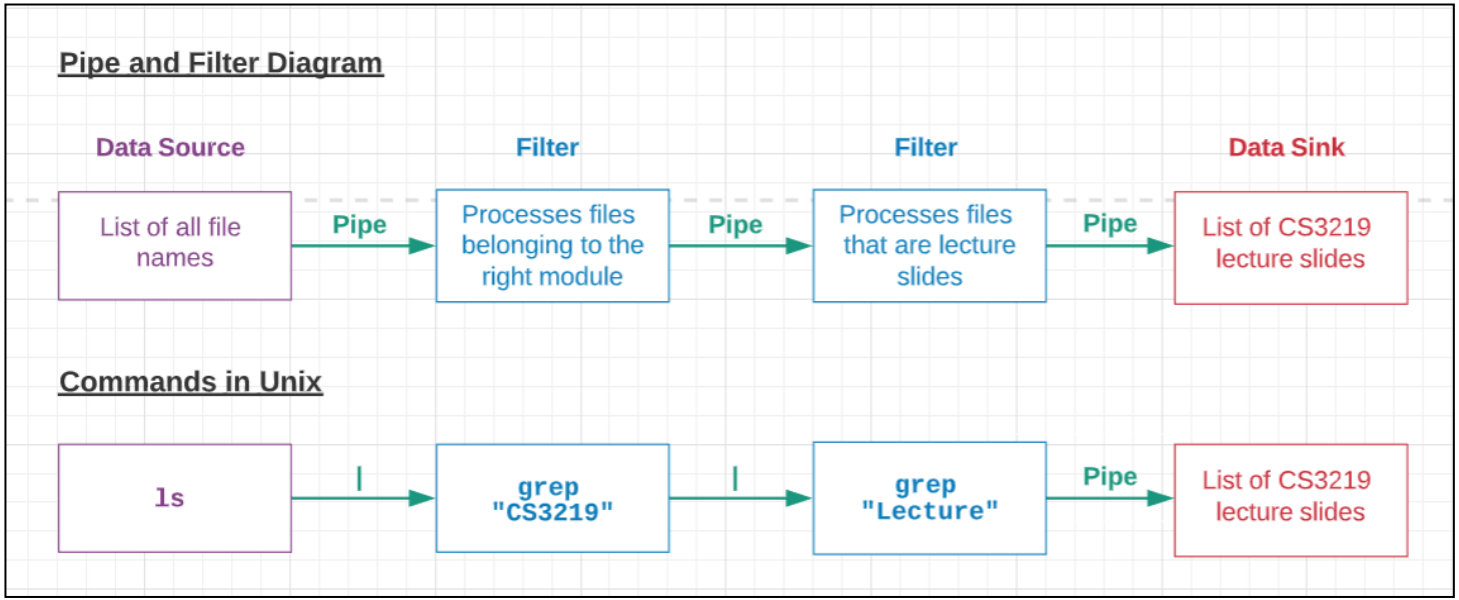


- **Catalog** should handle different searches and provide links to movie details, video clips, and an online store for buying photos, films, and videos.
- The **client program** is just a user interface, usually a web browser, that helps users access services.
- New servers can be added or upgraded easily without disrupting the rest of the system.

3.4 Pipe and Filter Architecture

- This model describes how a system processes inputs to produce outputs using **functional transformations**.
- Data flows from one to another and is transformed as it moves through the sequence.
- Each processing step is implemented as a transform.
- Input data flows through these transforms until converted to output.
- The ***transformations may execute sequentially or in parallel***.
- The ***data can be processed by each transform item by item or in a single batch***.
- The Pipe and Filter pattern comes from the **Unix system**, where **pipes** connect processes.
  - These pipes pass a text stream from one process to another.
- Systems following this model can be built using Unix commands, connected with pipes and controlled through the Unix shell.
- The term ***filter*** is used because a transformation "filters out" ***the data it can process from its input data stream***.

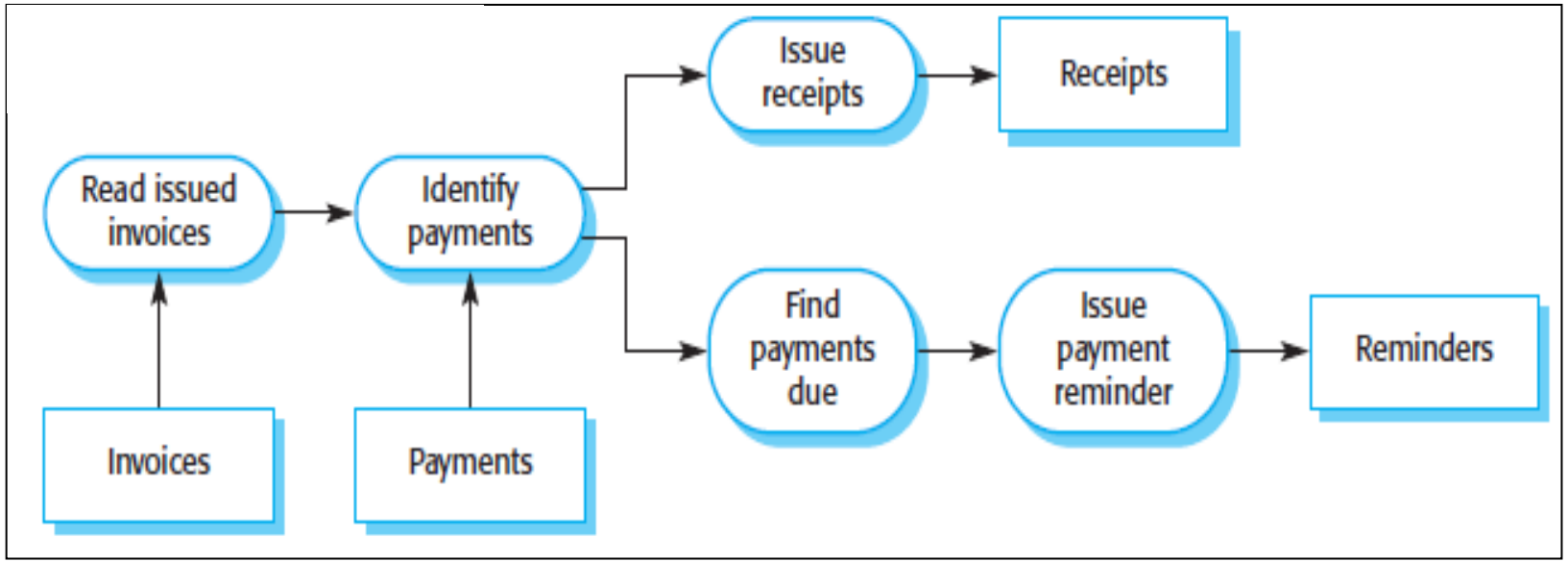

Filter
Filter
Data Source
Pipe
Image Enhancement
Pipe
Image Normalization
Pipe
Filter
Data Sink
Pipe
Bounding Box
Pipe
Object Detection


Pipe and Filter Diagram
Data Source
Filter
Filter
Data Sink
List of all file names
Pipe
Processes files belonging to the right module
Pipe
Processes files that are lecture slides
Pipe
List of CS3219 lecture slides
Commands in Unix
ls
|
grep "CS3219"
|
grep "Lecture"
Pipe
List of CS3219 lecture slides

Name	Repository
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and ***carries out one type of data transformation***. The data flows (as in a pipe) from one component to another for processing
Example	An example of pipe and filter system used for processing invoices is shown in next slide
When used	Commonly used in data-processing applications (both batch and transaction-based) where ***inputs are processed in separate stages to generate related outputs***.
Advantages	• Easy to understand and supports transformation reuse. • Workflow style matches the structure of many business processes. • Evolution by adding transformations is straightforward. • Can be implemented as either a sequential or concurrent system.
Disadvantages	• The way data is transferred must be agreed upon by both sides before communication happens. • Each transformation must read the input, process it, and convert the output into the required format. • This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

Pipe and Filter pattern

- The organization sends invoices to customers.
- Every week, they check which invoices have been paid:
  - **If paid** → A receipt is sent.
  - **If unpaid (past due date)** → A reminder is sent.



- Pipe and filter systems are best suited to batch processing systems and embedded systems where there is limited user interaction

4. Application Architectures

- Application architectures define the main features of a system.
- When building new systems, the same architecture can be reused.
- In business systems, this happens naturally when generic software is customized for new applications.
- As a software designer, you can **use models of application architectures** in a number of ways:

1. As a starting point for the architectural design process

- If you're unsure about the application type, start with a **general design**.
- Then, customize it to fit the specific system you're building.

2. As a design checklist

- After designing an application system, you can compare it with a **standard application architecture** to ensure they match and follow best practices.

3. As a way of organizing the work of the development team

- Application architectures define the stable structure of a system.
- Different parts can be developed at the same time, allowing team members to work on separate components.

4. As a means of assessing components for reuse

- If you have reusable components, you can compare them with common structures to check if they fit into your application's design.

5. As a vocabulary for talking about applications

- When discussing or comparing applications, you can use the generic architecture to explain how they work and relate to each other.

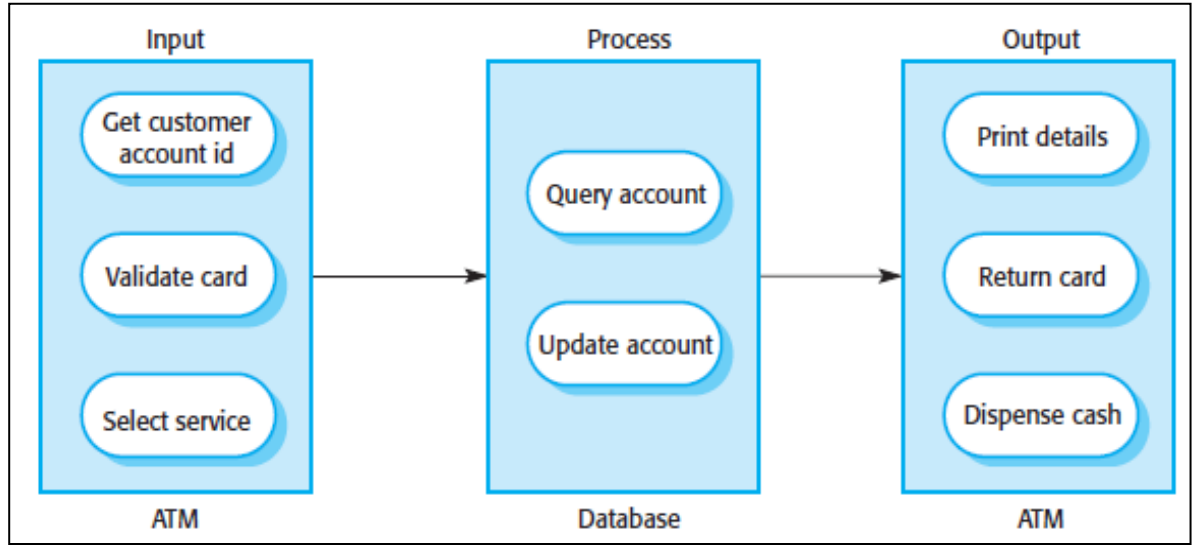
4.1 Transaction Processing Systems

- Transaction processing applications use databases to ***handle user requests and update information***.
- A **database transaction** is a set of operations treated as one unit (**atomic**).
  - ***All operations must be completed before changes are saved.***
  - ***If any operation fails, the database remains unchanged.***
- This prevents inconsistencies and ensures data accuracy.

An example of a database transaction is a customer request to withdraw money from a bank account using an ATM. This involves checking the customer account balance to see if sufficient funds are available, modifying the balance by the amount withdrawn and sending commands to the ATM to deliver the cash. Until all of these steps have been completed, the transaction is incomplete and the customer accounts database is not changed.

- **Transaction processing systems** handle user requests one at a time.
  - ✓ The user sends a request through an input system.
  - ✓ The system processes the request using specific logic.
  - ✓ A transaction is created and sent to the transaction manager (part of the database system).
  - ✓ The transaction manager ensures everything is completed correctly.
  - ✓ It then notifies the application that the process is finished.

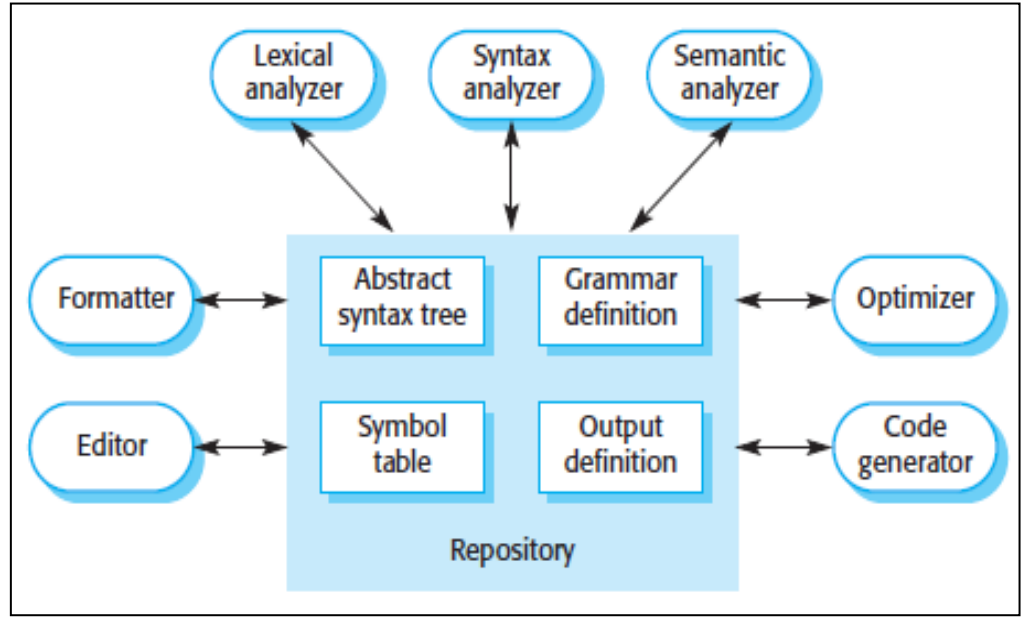
- Transaction processing systems may be organized as a ***"pipe and filter" architecture***, with system components responsible for input, processing, and output



4.2 Language processing systems

- Language processing systems ***translate one language into an alternative representation code of that language*** and, for programming languages, may also execute the resulting code.
- **Compilers** translate a programming language into machine code.
- **Natural Language Processing systems** may translate one natural language to another (Example : French to English)
- The source language instructions define the program to be executed, and a translator converts these into instructions for an abstract machine.
- These instructions are then interpreted by another component that fetches the instructions for execution and executes them using (if necessary) data from the environment.
- The output of the process is the result of interpreting the instructions on the input data.

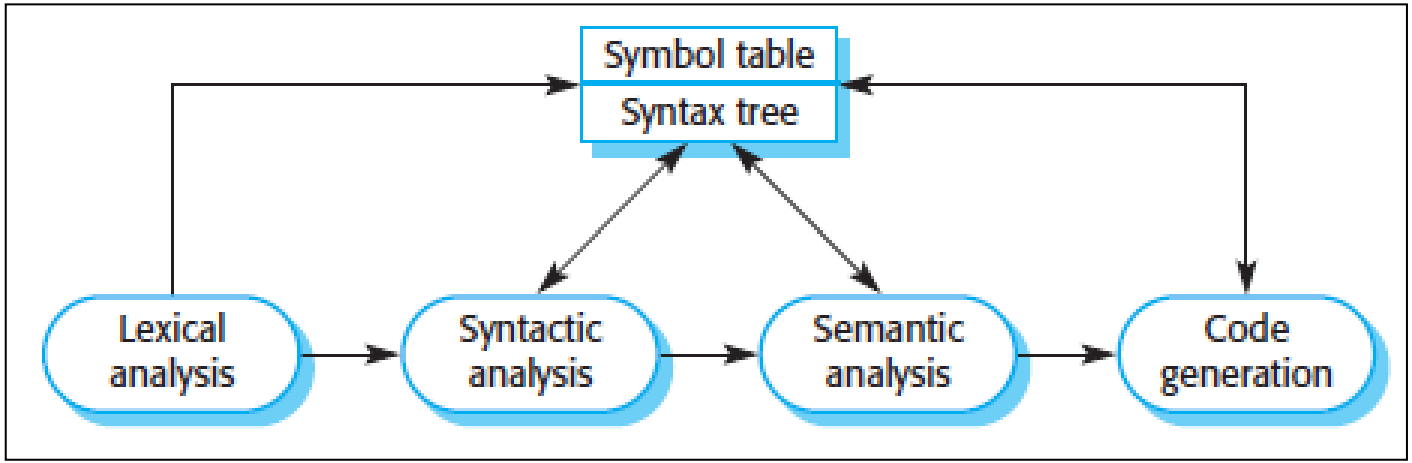
1. A **Lexical analyzer**, takes input language tokens and converts them into an internal form.
2. A **Symbol table**, which holds ***information about the names of entities*** (variables, class names, object names, etc.) used in the text that is being translated.
3. A **Syntax analyzer**, which checks the syntax of the language being translated.
- It uses a defined grammar of the language and builds a syntax tree.



Repository Architecture for a Language Processing System

4. A **Syntax tree**, which is an internal structure representing the program being compiled.
5. A **Semantic analyzer**, which uses information from the syntax tree and the symbol table to check the semantic correctness of the input language text.
6. A **Code generator**, which "walks" the syntax tree and generates abstract machine code.

- Compilers can be implemented using a composite of a **repository and a pipe and filter model.**
- In a compiler architecture, the ***symbol table is a repository for shared data***.
- The phases of lexical, syntactic, and semantic analysis are organized sequentially, and communicate through the shared symbol table.



A Pipe and Filter Compiler Architecture

Detailed Design Transaction Transformation

- A mapping technique, called ***structured design*** , is often characterized as a **data flow-oriented design** method because it provides a convenient ***transition from a data flow diagram to software architecture***.
- The transition from information flow (represented as a DFD) to program structure is accomplished as part of a six step process:

  (1) The type of information flow is established

  (2) Flow boundaries are indicated

  (3) DFD is mapped into the program structure

  (4) Control hierarchy is defined

  (5) Resultant structure is refined using design measures and heuristics

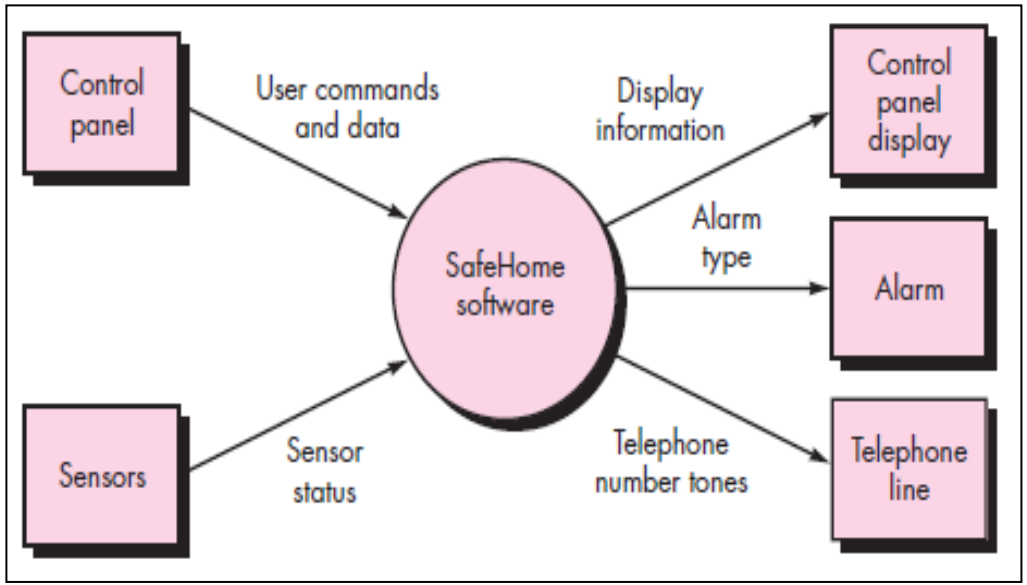
  (6) Architectural description is refined and elaborated.

1. Transform Mapping

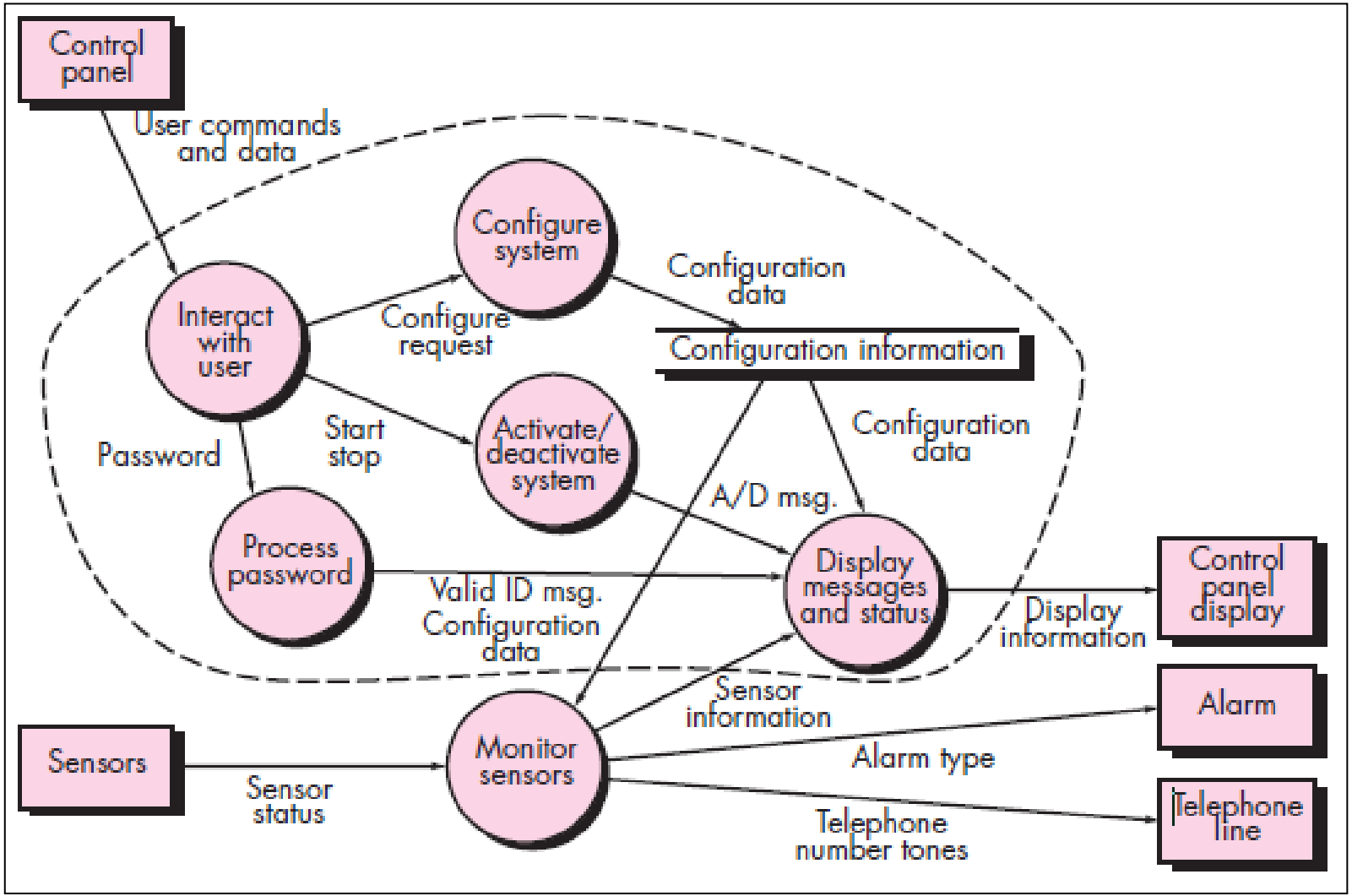
- Transform mapping is a design process that helps convert a Data Flow Diagram (DFD) with transform flow into a structured architecture.
- It follows specific steps to organize the system efficiently.
- One element of the analysis model is a set of data flow diagrams that describe information flow within the **security function**.
- The following design steps are initiated to map these data flow diagrams into a software architecture

Step 1. *Review the fundamental system model*

- The fundamental system model (or context diagram) provides a **high-level view** of a system's security function.
- It simplifies complex security mechanisms by representing them as a **single transformation process**, showing only external entities that interact with the system.



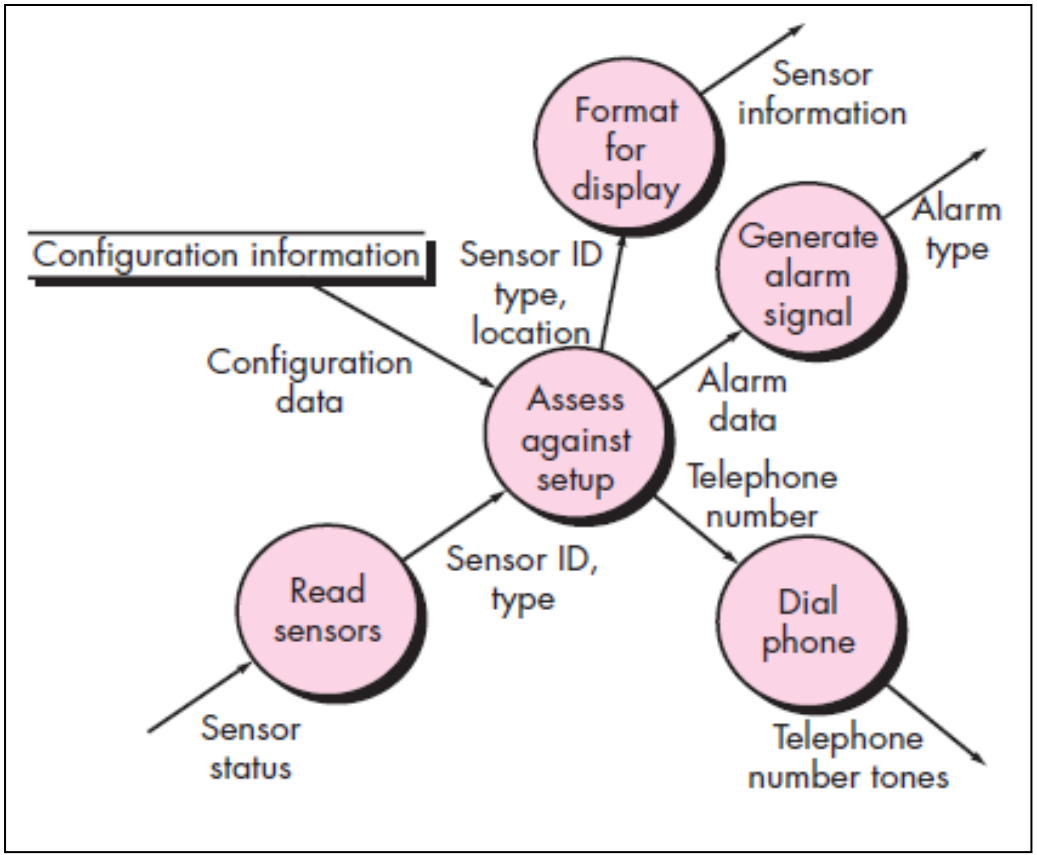
Context-level DFD_*SafeHome* security function



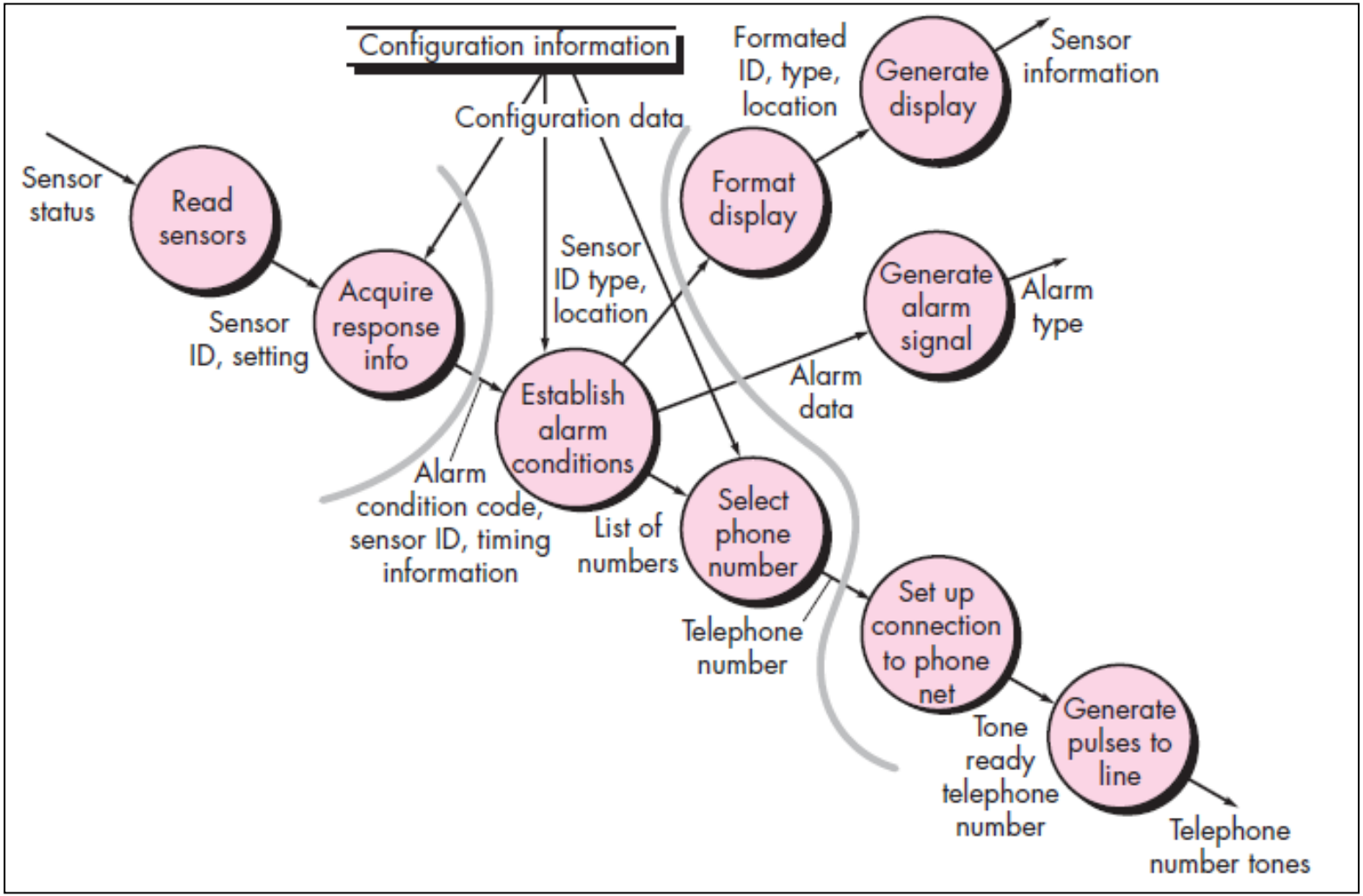
Level 1 DFD for the *SafeHome* security function

Step 2. Review and refine data flow diagrams for the software

- Information obtained from the ***requirements model is refined to produce greater detail***.
- For example, the level 2 DFD for ***monitor sensors*** is examined, and a level 3 data flow diagram is derived
- At **level 3**, each transform in the data flow diagram exhibits relatively **high cohesion**.
- The process implied by a transform performs a single, distinct function that can be implemented as a component in the *SafeHome* software



Level 2 DFD that refines the *monitor sensors* transform



Level 3 DFD for *monitor sensors* with flow boundaries

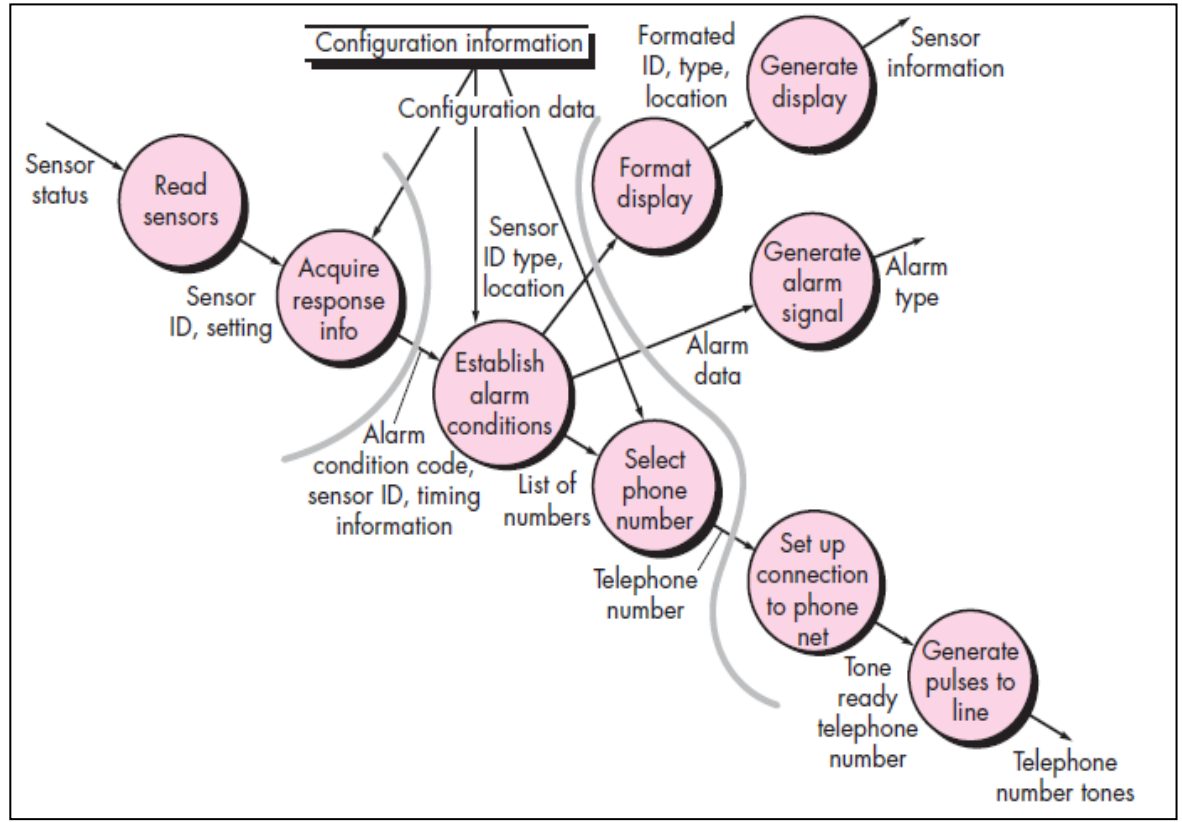
Step 3. Determine whether the DFD has transform or transaction flow characteristics

- Evaluating the Level 3 DFD for *monitor sensors* with flow boundaries , we ***see data entering the software along one incoming path and exiting along three outgoing paths***.
- An overall transform characteristic will be assumed for information flow.

Step 4. Isolate the transform center by specifying incoming and outgoing flow boundaries

- **Incoming data flows** along a path in which ***information is converted from external to internal form***
- **Outgoing flow** converts ***internalized data to external form***.
- Incoming and outgoing flow boundaries are open to interpretation.
- That is, different designers may select slightly different points in the flow as boundary locations.
- In fact, alternative design solutions can be derived by varying the placement of flow boundaries.
- Although care should be taken when boundaries are selected, a variance of one bubble along a flow path will generally have little impact on the final program structure.

- ***Flow boundaries*** are illustrated as shaded curves running vertically through the flow .
- The ***transforms*** (bubbles) that constitute the transform center lie within the two shaded boundaries that run from top to bottom.
- An argument can be made to readjust a boundary (e.g., an incoming flow boundary separating *read sensors* and *acquire response info* could be proposed).

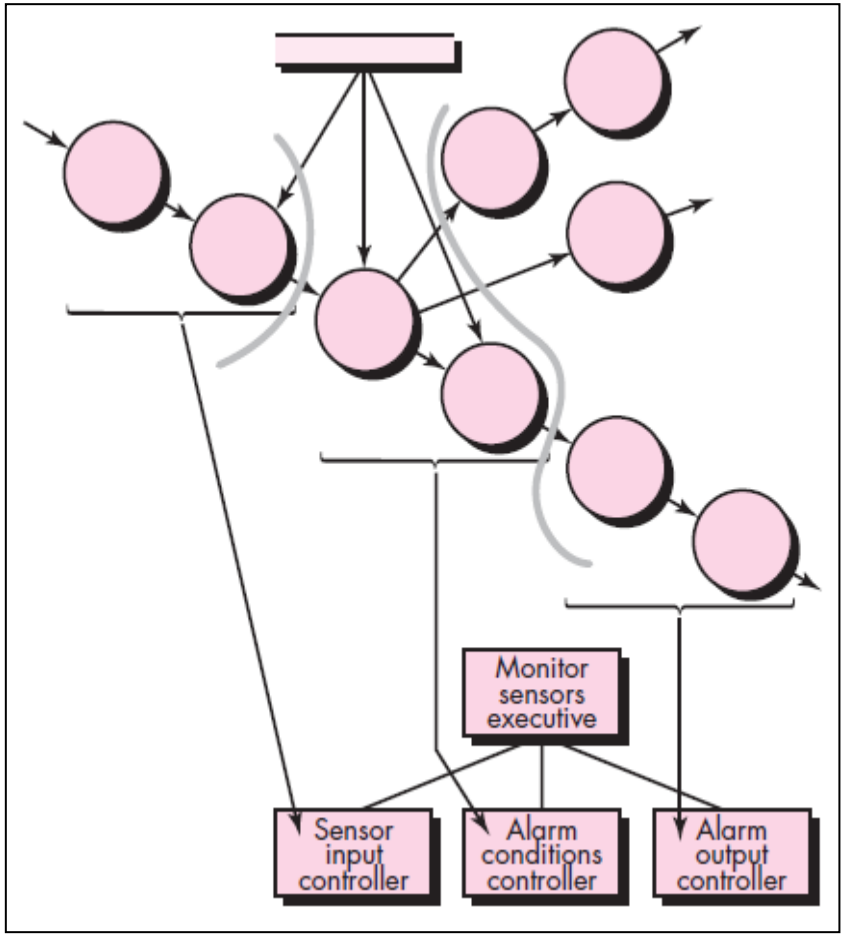


- The emphasis in this design step should be on selecting reasonable boundaries, rather than lengthy iteration on placement of divisions.

Step 5. Perform "first-level factoring."

- The program architecture derived using this mapping results in a top-down distribution of control.
- ***Factoring*** leads to a program structure in which
  - ***Top-level components*** perform decision making
  - ***Low level components*** perform most input, computation, and output work.
  - ***Middle-level components*** perform some control and do moderate amounts of work.
- When transform flow is encountered, a DFD is mapped to a specific structure (a call and return architecture) that provides control for incoming, transform, and outgoing information processing

- A **main controller** (called *monitor sensors executive*) resides at the top of the program structure and coordinates the following subordinate control functions:
  - An ***incoming information processing controller***, called ***sensor input controller***, coordinates receipt of all incoming data.
  - A ***transform flow controller***, called ***alarm conditions controller***, supervises all operations on data in internalized form (e.g., a module that invokes various data transformation procedures).
  - An ***outgoing information processing controller***, called ***alarm output controller***, coordinates production of output information.

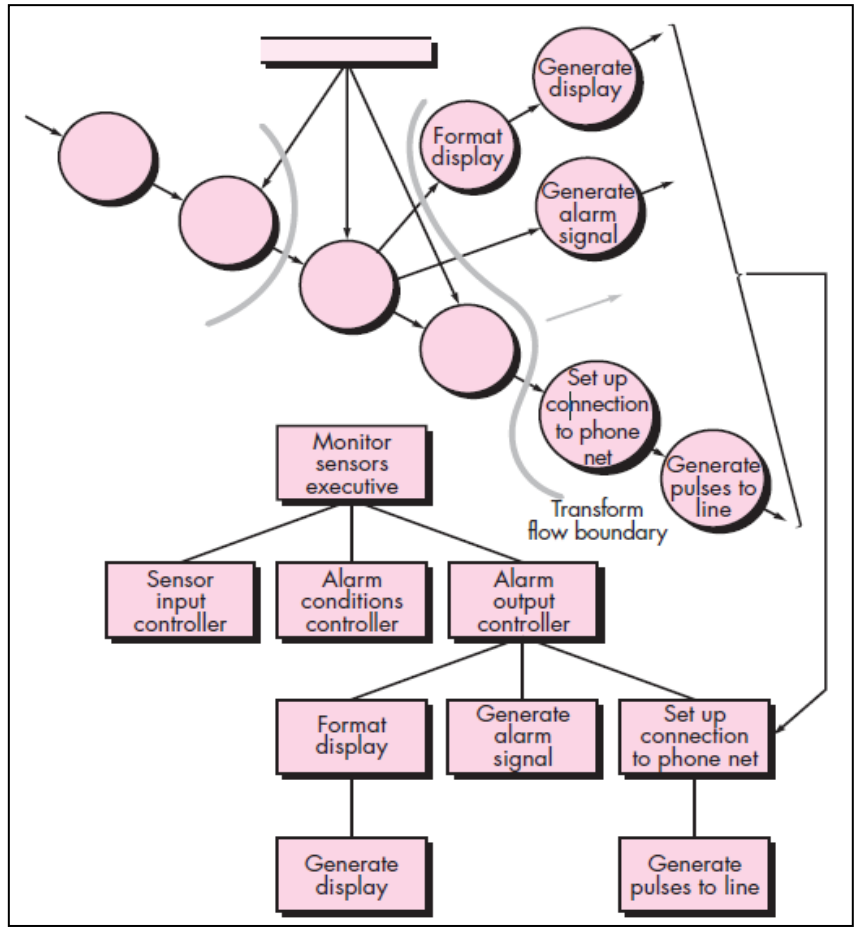


First-level factoring for *monitor Sensors*

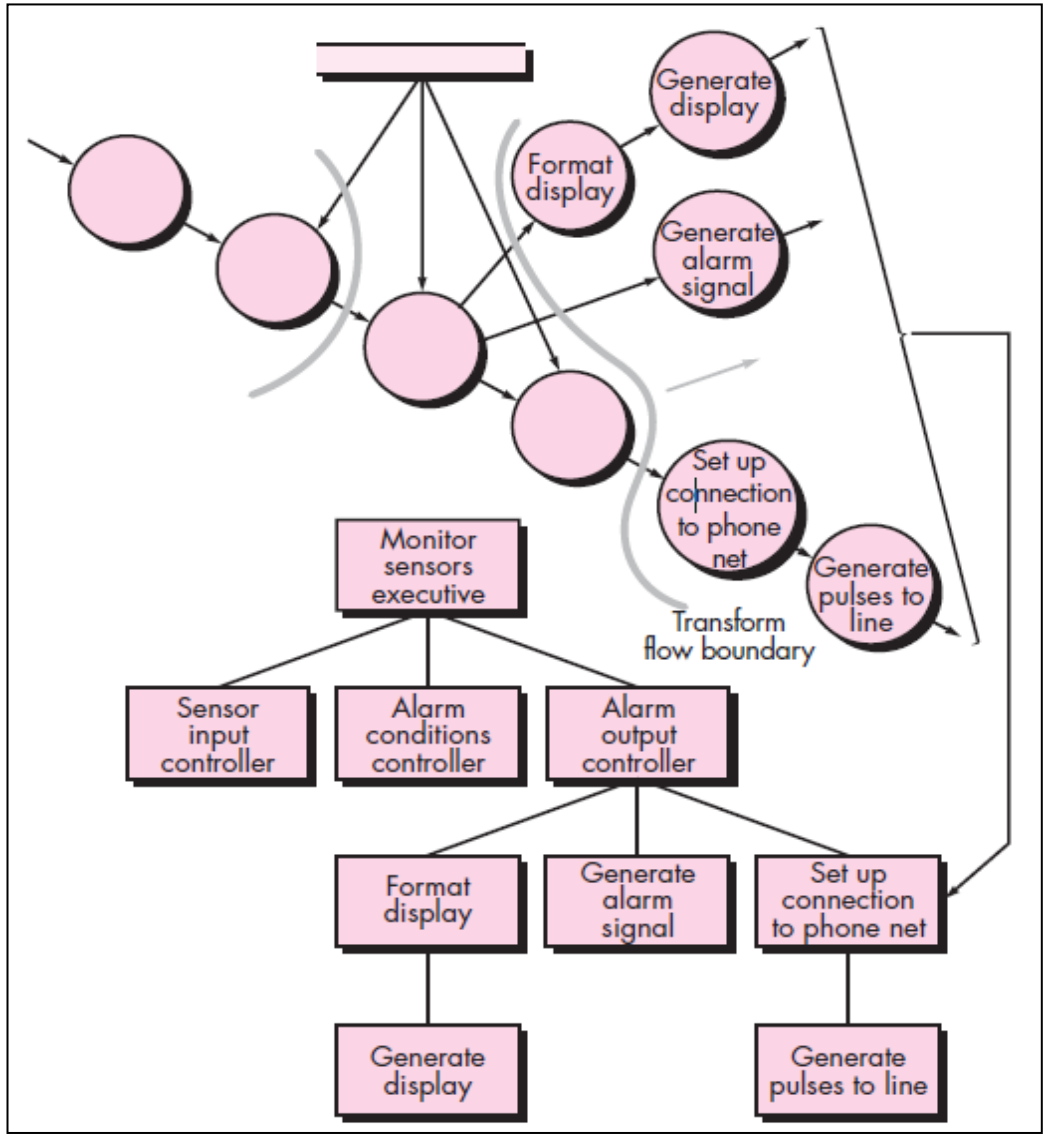
Step 6. Perform "second-level factoring."

- Second-level factoring is accomplished by ***mapping individual transforms (bubbles) of a DFD into appropriate modules within the architecture***.

- Beginning at the transform center boundary and moving outward along incoming and then outgoing paths, transforms are mapped into subordinate levels of the software structure.
- Figure illustrates a one-to-one mapping between DFD transforms and software modules, different mappings frequently occur.
- Two or even three bubbles can be combined and represented as one component, or a single bubble may be expanded to two or more components.

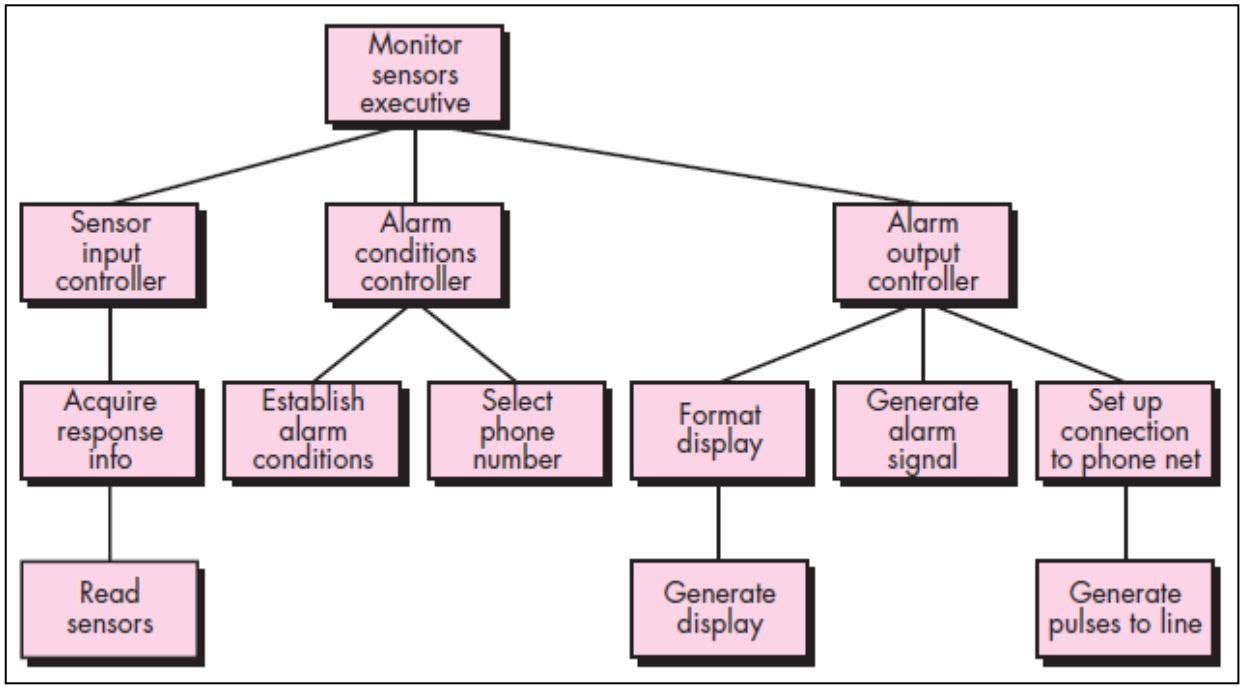


- Factoring is again accomplished by ***moving outward from the transform center boundary on the incoming flow side***.
- The transform center of monitor sensors subsystem software is mapped somewhat differently.
- Each of the data conversion or calculation transforms of the transform portion of the DFD is mapped into a module subordinate to the transform controller.



Second-level factoring for *monitor sensors*

- The components mapped in the preceding manner and shown in Figure represent an initial design of software architecture.
- Although components are named in a manner that implies function, a brief processing narrative (adapted from the process specification developed for a data transformation created during requirements modeling) should be written for each.
- The narrative describes the component interface, internal data structures, a functional narrative, and a brief discussion of restrictions and special features (e.g., file input-output, hardware dependent characteristics, special timing requirements).



First-iteration structure for *monitor sensors*

Step 7. Refine the first-iteration architecture using design heuristics for improved software quality.

- A first-iteration architecture can always be refined by applying concepts of functional independence .
- Components are exploded or imploded to produce sensible factoring, separation of concerns, good cohesion, minimal coupling, and most important, a structure that can be implemented without difficulty, tested without confusion, and maintained without grief.

OBJECT ORIENTED DESIGN

- In an object-oriented system, ***objects interact with each other***.
- Each object has its **own data** (state) and **functions** (operations).
- The data is private, meaning it can only be changed using the object's own functions.
- Object-oriented design focuses on creating object classes and defining how they relate to each other.
- When the program runs, objects are created automatically based on the class definitions.
- To design a system from concept to detailed, object-oriented design, follow these steps:
  1. ***Understand and define the context and the external interactions with the system.***
  2. ***Design the system architecture.***
  3. ***Identify the principal objects in the system.***
  4. ***Develop design models.***
  5. ***Specify interfaces.***

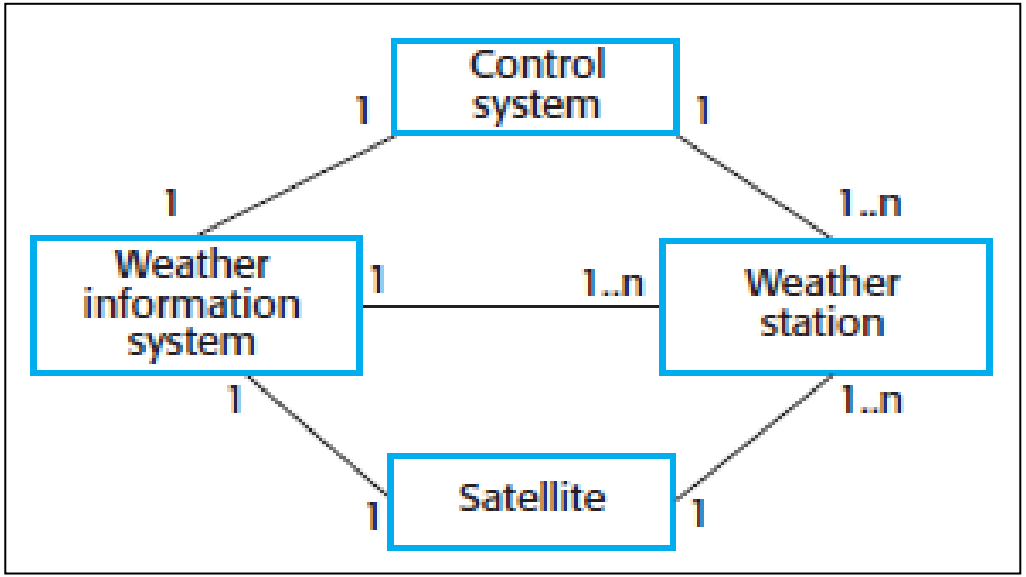
1. System Context and Interactions

- The first step in software design is understanding how the software interacts with its environment.
- This helps decide what the system should do and how it should connect with other parts.
- **System context models** and **interaction models** show how a system connects with its environment in different ways:

  1. A **System Context Model** shows how the system being developed interacts with other systems in its environment.
  2. An **Interaction Model** is a ***dynamic model*** that shows how the system interacts with its environment as it is used.

- **Context model** of a system may be ***represented using associations***.
- Associations simply show that there are some relationships between the entities involved in the association.
- The environment of the system using can be documented using a ***simple block diagram***, showing the entities in the system and their associations.

- The systems in the environment of each **weather station** are a ***weather information system***, an ***onboard satellite system***, and a ***control system***.
- The ***cardinality*** information on the link shows that there is a single control system but several weather stations, one satellite, and one general weather information system.



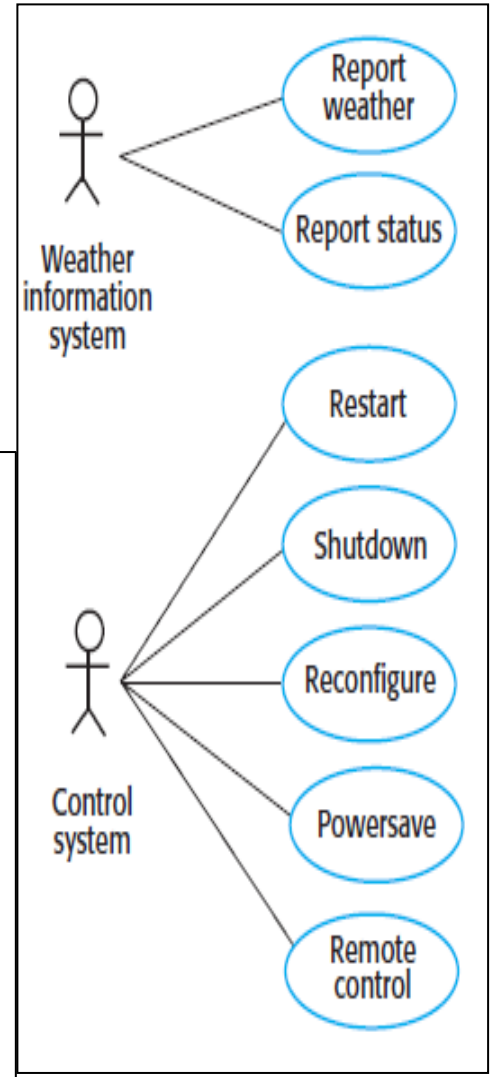
System context for the weather station

- Each use case represents an interaction with the system

System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Data	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum and average wind speeds; the total rainfall; and the wind direction as sampled at 5-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour, but this frequency may differ from one station to another and may be modified in future.

Use case description—Report weather

- In UML Use Case Diagrams, each action (interaction) is written inside ***an ellipse***, and the user or external system involved is shown as a ***stick figure***
- Weather station interacts with the weather information system to report weather data and the status of the weather station hardware.
- A control system do the interactions that can issue specific weather station control commands.

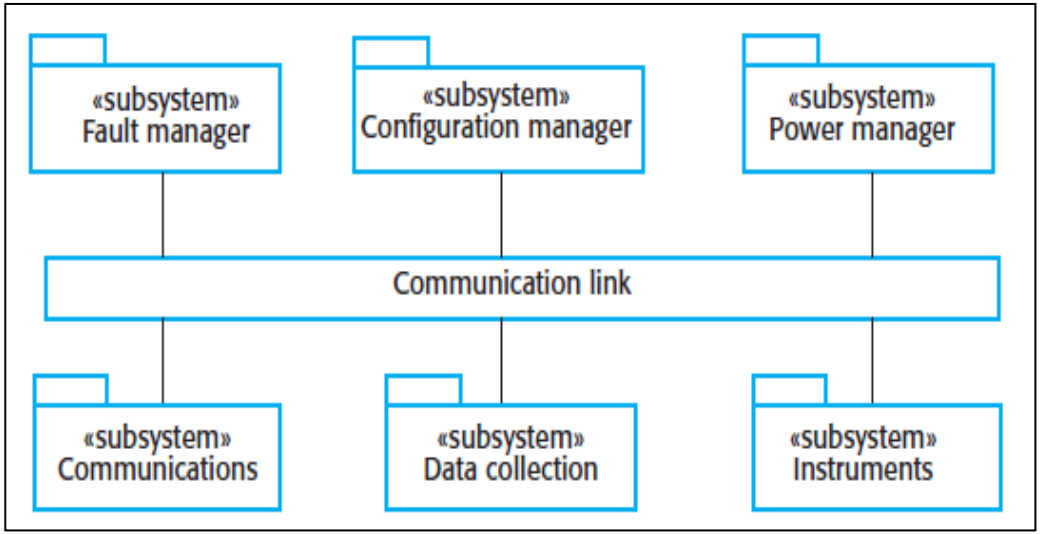


Weather station use cases

Report weather	: Send weather data to the weather information system
Report status	: Send status information to the weather information system
Restart	: If the weather station is shut down, restart the system
Shutdown	: Shut down the weather station
Reconfigure	: Reconfigure the weather station software
Powersave	: Put the weather station into power-saving mode
Remote control	: Send control commands to any weather station subsystem

2. Architectural Design

- The way the software interacts with its environment helps in designing the system's architecture.
- The **weather station** is ***composed of independent subsystems*** that communicate by broadcasting messages on a common infrastructure, shown as ***Communication link***.
- Each subsystem listens for messages on that infrastructure and picks up the messages that are intended for them.



High-level architecture of weather station

- This **"listener model"** is a commonly used architectural style for distributed systems.
- When the system gets a **shutdown command**, all subsystems receive it and shut down properly in the right order.

3. Object Class Identification

- The **use case description** helps to ***identify objects and operations in the system***.
- To implement the **Report Weather** use case, you need:

  Instrument objects – Collect weather data (e.g., temperature, humidity).

  Weather summary object – Stores and processes the collected data.
- There are different ways to find object classes in object-oriented systems.

  1. Use **tangible entities** (things) in the
  - ***Application domain such as aircraft***
  - ***Roles such as manager***
  - ***Events such as request***
  - ***Interactions such as meetings***
  - ***Locations such as offices***
  - ***Organizational units such as companies***

  2. Use a **scenario-based analysis** where various scenarios of system use are identified and analyzed in turn.

  - As each scenario is analyzed, the team responsible for the analysis must ***identify the required objects, attributes, and operations***.

Just as with nouns, some verbs may obviously not make it into the design, but be thorough.

You may want to highlight the same verb more than once if it appears in different contexts.

You are designing a restaurant reservation system. Each restaurant has information about its location, type of food, hours of operation, and reservations. The restaurants are broken up into regular, catering, and take-out where making a reservation means a different thing for each category of restaurant. Diners will be able to use the system either when logged into their account or as a guest. Diners, either as guests or logged in users, can make reservations. Reservations will be saved to a user's account if they are logged in. Logged in users can also view, modify, or delete reservations in their account. A user's account will store a name, rewards points, and their upcoming and past reservations. Restaurant users will have an account which can view, modify, or delete any reservation made for their restaurant. The restaurant user can also complete a reservation when diners arrive. All users will authenticate using a password or Google login.

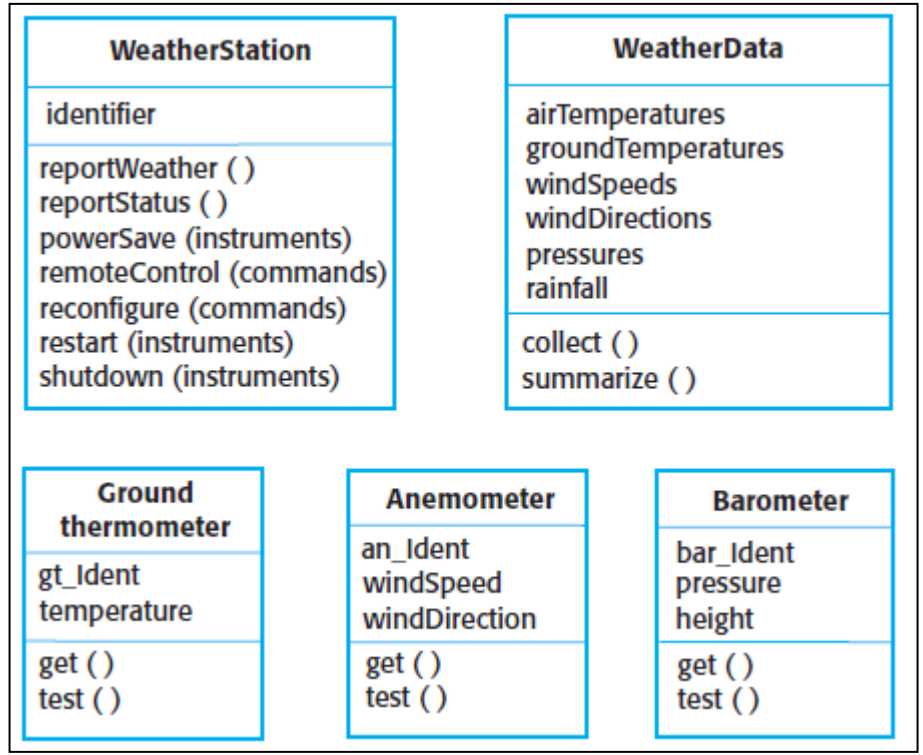
3. Use a **grammatical analysis** of a natural language description of the system to be constructed.

- ***Objects and attributes*** are **nouns**
- ***Operations or services*** are **verbs**.

Nouns	Verbs
restaurant [regular, catering, take-out] (location, type of food, hours of operation, reservations)	
reservation	
system	
liformation	
location	
hours of operation	
User [logged in, guest] (name, rewards points, password, Google login, reservations)	complete (reservation) saves (reservation) views/modifies/deletes (reservation) authenticate (password) authenticate (Google login)

- The ***Ground thermometer***, ***Anemometer***, and ***Barometer*** objects are **application domain objects**, and the ***WeatherStation*** and ***WeatherData*** objects have been identified from the **system description** and the **scenario (use case) description**.

- **WeatherStation** object class provides the ***basic interface of the weather station with its environment***.
- **WeatherData** object class is ***responsible for processing the report weather command***. It sends the summarized data from the weather station instruments to the weather information system.
- **Ground thermometer**, **Anemometer**, and **Barometer** object classes are ***directly related to instruments in the system***.



Weather station objects

- They reflect tangible hardware entities in the system and the operations are concerned with controlling that hardware

4. Design Models

- **Design or system models** show the objects or classes in a system and how they are connected.
- They act as a bridge between system requirements and actual implementation.
- When using **UML** for design, two types of models are created:

1. *Structural models*

- The static structure of a system shows how different object classes are connected.
- It includes:
  - **Generalization (Inheritance)** – One class inherits from another.
  - **Uses/Used-by** – One class depends on another.
  - **Composition** – One class is made up of other classes.
- These relationships help define how parts of the system interact and organize the design.

2. *Dynamic models*

- The dynamic structure of a system shows how objects interact at runtime. It includes:
  - The order of service requests between objects.
  - State changes that happen due to these interactions.
- This helps understand how the system behaves during execution.

- Three useful **UML model types** for detailing use case and architecture:

1. Subsystem Models

- Subsystem models ***group related objects*** into meaningful subsystems.
- These are represented using a form of **class diagram** with each subsystem shown as a package with enclosed objects.
- Subsystem models are ***structural*** models.

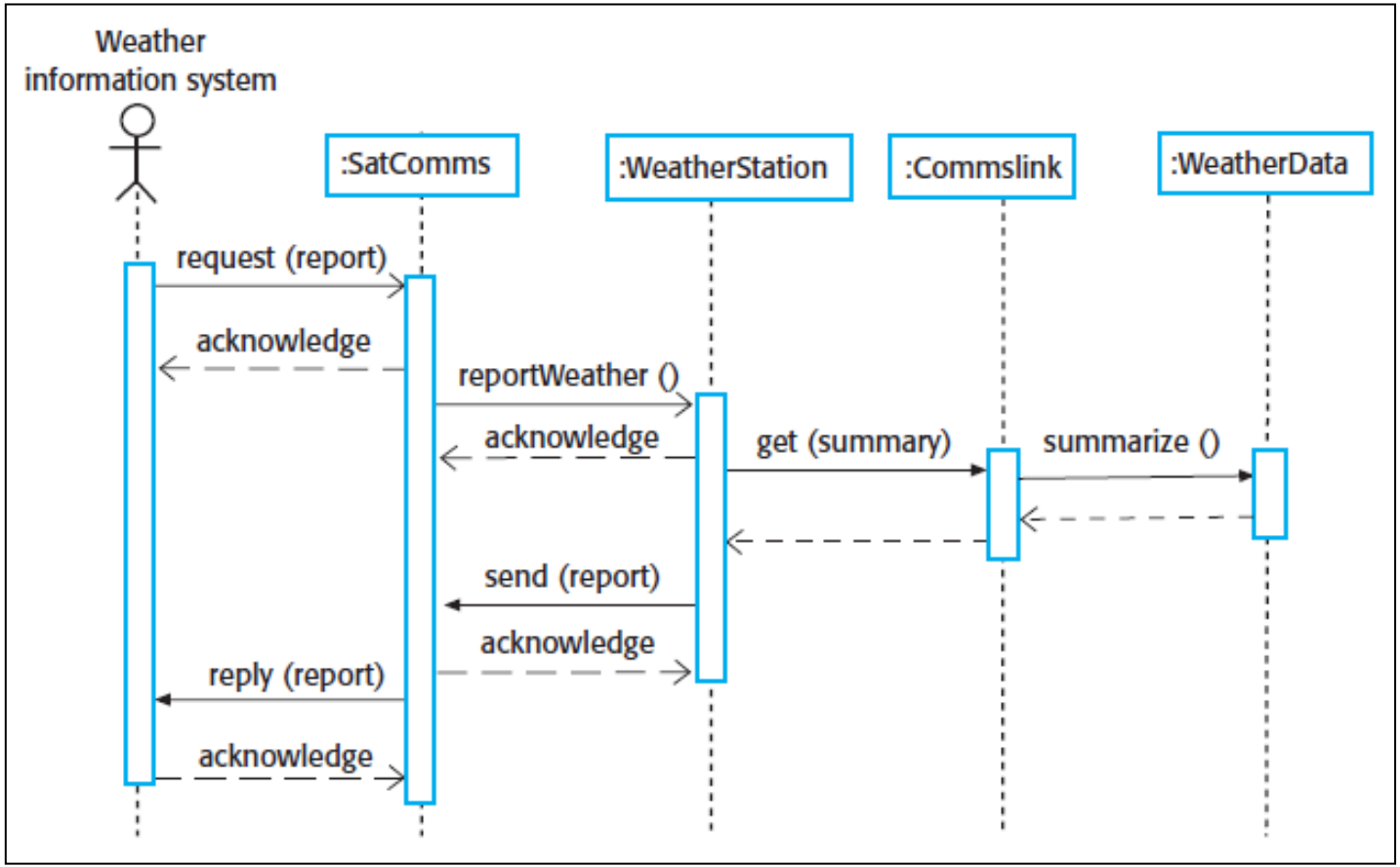
2. Sequence Models

- Show the ***sequence of object interactions***.
- These are represented using a UML **sequence** or a **collaboration diagram**.
- Sequence models are ***dynamic*** models.

3. State Machine Models

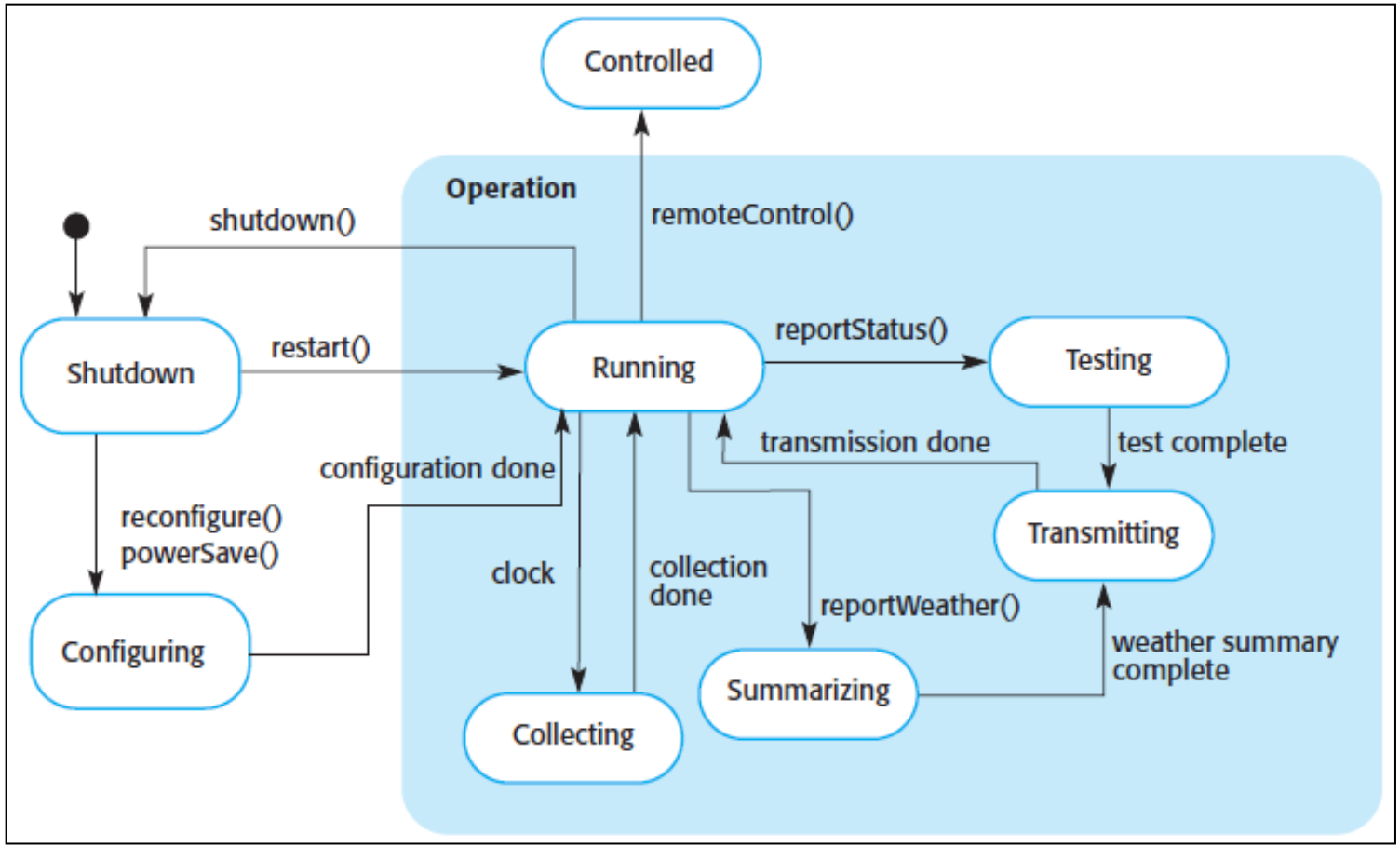
- Show ***how objects change their state*** in response to events.
- These are represented in the UML using **state diagrams**.
- State machine models are ***dynamic*** models.

- **Sequence models** are dynamic models that describe, for each mode of interaction, the sequence of object interactions that take place.
- This diagram shows the sequence of interactions that take place when an external system requests the summarized data from the weather station.



Sequence diagram describing data collection

- A **state diagram** for the weather station system that shows how it responds to requests for various services



Weather station state diagram

User-Interface Design

1. The Golden Rules

a. Place the User in Control

- Simplify mode of interaction where design should allow user to maintain control.
- Mandel defines a number of **design principles** that allow the user to maintain control:

(i) Define interaction modes in a way that does not force a user into unnecessary or undesired actions.

Example:
If spell check is selected in a word-processor menu, the software moves to a spell-checking mode. There is no reason to force the user to remain in spell-checking mode if the user desires to make a small text edit along the way

Example : Software might allow a user to interact via keyboard commands, mouse movement, a digitizer pen, a multitouch screen, or voice recognition commands.
But every action is not responsive to every interaction mechanism.

(iii) Allow user interaction to be interruptible and undoable.

Even when involved in a sequence of actions, the user should be able to interrupt the sequence to do something else (without losing the work that had been done).
The user should also be able to "undo" any action.

(iv) Streamline interaction as skill levels advance and allow the interaction to be customized

Users often find that they perform the same sequence of interactions repeatedly.

(v) Hide technical internals from the casual user.

The user should not be aware of the operating system, file management functions, or other arcane computing technology.

(vi) Design for direct interaction with objects that appear on the screen.

An application interface that allows a user to "stretch" an object (scale it in size) is an implementation of direct manipulation

b. Reduce the User's Memory Load

- Mandel defines design principles that enable an interface to reduce the user's memory load:

  (i) Reduce demand on short-term memory

  – The interface should be designed to ***reduce the requirement to remember past actions, inputs, and results***.

  – This can be accomplished by providing visual indications that enable a user to recognize past actions, rather than having to recall them.

  (ii) Establish meaningful defaults

  – The ***initial set of defaults should make sense for the average user***, but a user should be able to specify individual preferences

  (iii) Define shortcuts that are intuitive

  – When mnemonics are used to accomplish system function (e.g., alt-P to invoke the print function), the ***mnemonic should be tied to the action in a way that is easy to remember*** (e.g., first letter of the task to be invoked).

(iv) The visual layout of the interface should be based on a real-world image.

- Example: A bill payment system should use a checkbook and check register metaphor to guide the user through the bill paying process.
- This enables the user to rely on well-understood visual cues, rather than memorizing an deep interaction sequence.

(v) Disclose information in a progressive fashion

- The interface should be organized hierarchically.
- Information about a task, an object, or some behavior should be presented first at a high level of abstraction.
- More detail should be presented after the user indicates interest with a mouse pick.
- Example : Word-processing applications, is the underlining function.
- The function itself is one of a number of functions under a *text style* menu.
- However, every underlining capability is not listed. The user must pick underlining; then all underlining options (e.g., single underline, double underline, dashed underline) are presented.

c. Make the Interface Consistent

- All visual information is organized according to design rules that are maintained throughout all screen displays.
- Input mechanisms are constrained to a limited set that is used consistently throughout the application.
- Mechanisms for navigating from task to task are consistently defined and implemented.
- Mandel defines a set of design principles that help make the interface consistent:

(i) Allow the user to put the current task into a meaningful context.

– Ensuring users have indicators, such as window titles, graphical icons, and consistent color coding, is crucial for providing clarity about the current context of their tasks.

(ii) Maintain consistency across a family of applications.

– A set of applications (or products) should all implement the same design rules so that consistency is maintained for all interaction.

(iii) If past interactive models have created user expectations, do not make changes unless there is a compelling reason to do so

– Example: The use of alt-S to save a file), the user expects this in every application he encounters.

– A change (e.g., using alt-S to invoke scaling) will cause confusion.

2. User Interface Analysis and Design

- The overall process for analyzing and designing a user interface begins with the ***creation of different models of system function***.
- Tools are used to prototype and ultimately implement the design model, and the result is evaluated by end users for quality.
- **Four different models** come into play when a user interface is to be analyzed and designed.

  (i) ***User Model*** *:* Establishes by a human engineer or the software engineer

  (ii) ***Design Model****:* Created by software engineer

  (iii) ***User's Mental Model* or *System Perception*** *:* A conceptual image is developed by the end user.

  (iv) ***Implementation Model*** *:* Responsible for implementing the system carry out the creation process.

(i) User Model

- The user model establishes the ***profile of end users of the system***.
- To build an effective user interface, all design should begin with an understanding of the intended users, including ***Profiles of their age, Gender, Physical abilities, Education, Cultural or ethnic background, Motivation, Goals and Personality.***
- Users can be categorized as:

  (i) Novices

  – No syntactic knowledge of the system and little semantic knowledge of the application or computer usage in general.

  (ii) Knowledgeable, intermittent users

  – Reasonable semantic knowledge of the application but relatively low recall of syntactic information necessary to use the interface.

  (iii) Knowledgeable, frequent users

  – Good semantic and syntactic knowledge that often leads to the "power-user syndrome"; that is, individuals who look for shortcuts and abbreviated modes of interaction.

(ii) User's Mental Model

- The user's mental model (system perception), refers to the ***conceptual image of the system*** that users hold in their minds.

> ***When someone using a word processor is asked to explain how it works, their response will be influenced by how they perceive the system. The accuracy of their description will vary based on their experience level (e.g., beginners might give a basic explanation). Someone who knows word processors well but has only used a specific one briefly might give a better description than a novice who has spent more time learning it.***

(iii) Implementation model

- The implementation model combines the ***external appearance of the computer- based system*** (the look and feel of the interface), coupled with all ***supporting information*** (books, manuals, videotapes, help files) that **describes interface syntax and semantics**.
- When the implementation model and the user's mental model are coincident, users generally feel comfortable with the software and use it effectively.
- It must accurately reflect syntactic and semantic information about the interface.

The Process

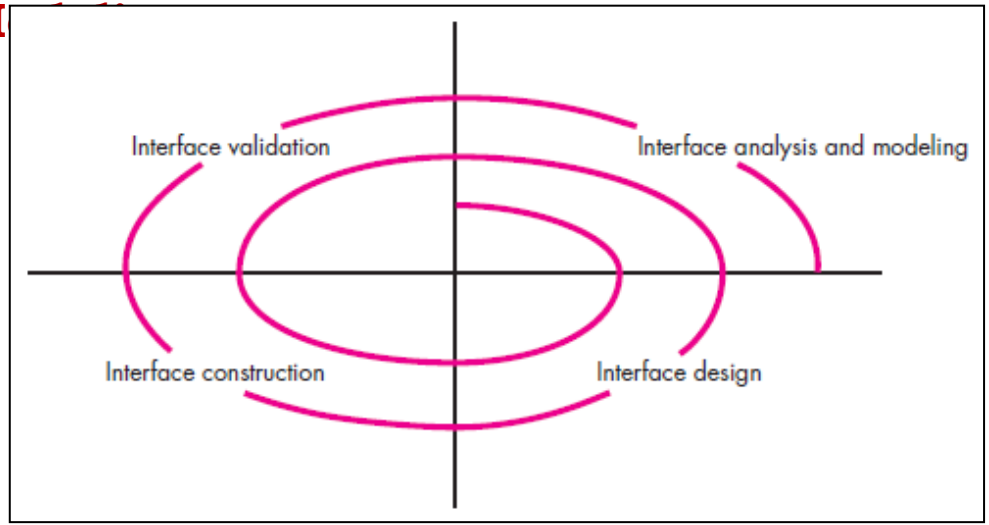
- The analysis and design process for user interfaces is ***iterative*** and can be ***represented using a spiral model***.
- The user interface analysis and design process begins at the interior of the spiral and encompasses four distinct framework activities:

1. Interface Analysis and M

2. Interface Design

3. Interface Construction

4. Interface Validation



Interface analysis

- Focuses on the ***profile of the users who will interact with the system.***
- Skill level, business understanding, and general interest to the new system are recorded
- Different user categories are defined.
  - For each user category, requirements are elicited.

- Once general requirements have been defined, a more detailed ***task analysis*** is conducted.
- Those tasks that the user performs to accomplish the goals of the system are identified, described, and elaborated
- Finally, analysis of the user environment focuses on the physical work environment.
- Among the questions to be asked are

- ***Where will the interface be located physically?***
- ***Will the user be sitting, standing, or performing other tasks unrelated to the interface?***
- ***Does the interface hardware accommodate space, light, or noise constraints?***
- ***Are there special human factors considerations driven by environmental factors?***

1. INTERFACE ANALYSIS

- A key rule of all software engineering process models is "***understand the problem before you attempt to design a solution***".
- In user interface design, understanding the problem means understanding

(1) People (end users) who will interact with the system through the interface

(2) Tasks that end users must perform to do their work

(3) Content that is presented as part of the interface

(4) Environment in which these tasks will be conducted

a. User Analysis

- Understanding the user is crucial before examining into technical aspects, especially when it comes to the user interface.
- ***Each user has their own perception of the software, which may differ from others' and even from the software engineer's design.***
- To bridge this gap, it's essential to comprehend the users and how they will interact with the system.

- The following set of questions will help you to better understand the users of a system:
  - *Are users trained professionals, technicians, clerical, or manufacturing workers?*
  - *What level of formal education does the average user have?*
  - *Are the users capable of learning from written materials or have they expressed a desire for classroom training?*
  - *Are users expert typists or keyboard phobic?*
  - *What is the age range of the user community?*
  - *Will the users be represented predominately by one gender?*
  - *How are users compensated for the work they perform?*
  - *Do users work normal office hours or do they work until the job is done?*
  - *Is the software to be an integral part of the work users do or will it be used only occasionally?*
  - *What is the primary spoken language among users?*
  - *What are the consequences if a user makes a mistake using the system?*
  - *Are users experts in the subject matter that is addressed by the system?*
  - *Do users want to know about the technology that sits behind the interface?*

b. Task Analysis and Modeling

- The goal of task analysis is to answer the following questions:
  - *What work will the user perform in specific circumstances?*
  - *What tasks and subtasks will be performed as the user does the work?*
  - *What specific problem domain objects will the user manipulate as work is performed?*
  - *What is the sequence of work tasks—the workflow?*
  - *What is the hierarchy of tasks?*

Use cases

- Use case outlines ***how a person interacts with a system***, particularly in user interface design.
- Use cases are written informally, often as a simple paragraph, from the user's perspective.

Assume that a small software company wants to build a computer-aided design system explicitly for interior designers

I begin by sketching the floor plan of the room, the dimensions and the location of windows and doors. I'm very concerned about light as it enters the room, about the view out of the windows (if it's beautiful, I want to draw attention to it), about the running length of an unobstructed wall, about the flow of movement through the room. I then look at the list of furniture my customer and I have chosen—tables, chairs, sofa, cabinets, the list of accents—lamps, rugs, paintings, sculpture, plants, smaller pieces, and my notes on any desires my customer has for placement. I then draw each item from my lists using a template that is scaled to the floor plan. I label each item I draw and use pencil because I always move things. I consider a number of alternative placements and decide on the one I like best. Then, I draw a rendering (a 3-D picture) of the room to give my customer a feel for what it'll look like.

- This use case provides a basic description of one important work task for the computer-aided design system.
- From it, you can extract tasks, objects, and the overall flow of the interaction

Task Elaboration

- Task analysis for interface design uses an elaborative approach to assist in ***understanding the human activities the user interface must accommodate***.
- Task analysis can be applied in two ways.

(1) When ***transitioning from manual to computer-based systems***, it's essential to understand the tasks people currently do manually and translate them into tasks suitable for the new system.

(2) You can ***analyze an existing computer-based solution's specifications*** to derive user tasks that align with the user model, design model, and system perception.

- The initial step involves defining and categorizing tasks which can be done through stepwise elaboration method.
- When you observe **an interior designer working**, you see that interior design involves several main activities:
  - ***Furniture layout***
  - ***Fabric and Material selection***
  - ***Wall and Window coverings selection***
  - ***Presentation (to the customer)***
  - ***Costing and Shopping.***

- Each of these ***main tasks can be broken down into smaller subtasks***.
- **Furniture layout** can be refined into the following tasks:

  1. *Draw a floor plan based on room dimensions*
  2. *Place windows and doors at appropriate locations*

  3a. *Use furniture templates to draw scaled furniture outlines on the floor plan*

  3b. *Use accents templates to draw scaled accents on the floor plan*

  4. *Move furniture outlines and accent outlines to get the best placement*
  5. *Label all furniture and accent outlines*
  6. *Draw dimensions to show location*
  7. *Draw a perspective-rendering view for the customer.*

Object Elaboration

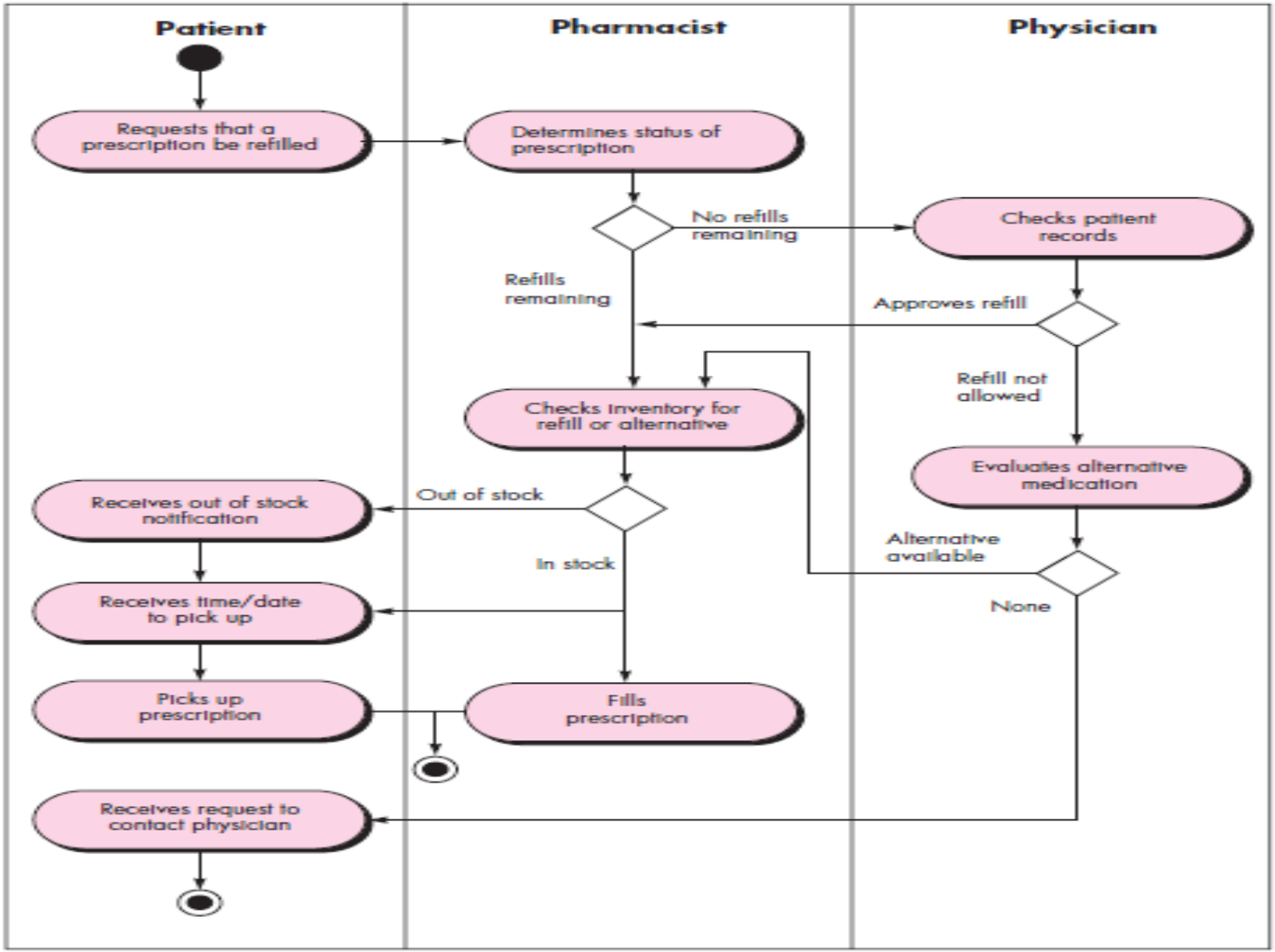
- Instead of concentrating solely on user tasks, you can look at the ***use case and other user-provided information*** to ***identify the physical objects*** utilized by the interior designer.
- These ***objects can be categorized into classes***.
- **Attributes** of each class are defined, and an evaluation of the actions applied to each object provide a list of operations
- **Example** : **Furniture template** might translate into a class called **Furniture** with **attributes** (size, shape, location).
- The **interior designer** would
  - ***select*** the object from the **Furniture** class
  - ***move*** it to a position on the floor plan (another object in this context)
  - ***draw*** the furniture outline
- The tasks ***select***, ***move***, and ***draw*** are operations.

Workflow Analysis

- When multiple users with varying roles interact with a user interface, it may be necessary to go beyond task analysis and object elaboration and employ workflow analysis.
- This method helps you grasp ***how a work process explains when multiple people and roles are engaged***.

Imagine a company planning to automate the process of prescribing and delivering prescription drugs entirely. This process will be centered around a web-based application accessible to physicians (or their assistants), pharmacists, and patients.

- Workflow can be represented effectively with a UML ***swimlane diagram***.
- We focus solely on a small segment of the work process: **the scenario when a patient requests a refill.**
- A swimlane diagram that indicates the tasks and decisions for each of the three roles


Patient
Pharmacist
Physician
Requests that a prescription be refilled
Determines status of prescription
No refills remaining
Checks patient records
Refills remaining
Approves refill
Refill not allowed
Checks inventory for refill or alternative
Evaluates alternative medication
Out of stock
Receives out of stock notification
Alternative available
In stock
Receives time/date to pick up
None
Picks up prescription
Fills prescription
Receives request to contact physician

Hierarchical Representation

- A process of elaboration occurs as you begin to analyze the interface.
- Once the workflow is determined, you can outline a ***task hierarchy for each type of user.***
- The hierarchy is derived by a ***stepwise elaboration of each task*** identified for the user.
- Example, consider the following user task and subtask hierarchy.

User task: ***Requests that a prescription be refilled***

- ***Provide identifying information.***
  - **Specify name.**
  - **Specify userid.**
  - **Specify PIN and password.**
- ***Specify prescription number.***
- ***Specify date refill is required.***

c. Analysis of Display Content

- The user tasks identified lead to the presentation of a variety of different types of content.
- For modern applications, display content can range from
  - ***Character-based reports*** (e.g., a spreadsheet)
  - ***Graphical displays*** (e.g., a histogram, a 3-D model, a picture of a person)
  - ***Specialized information*** (e.g., audio or video files).
- The analysis modeling techniques identify the output data objects that are produced by an application.
- These data objects may be

  1. Generated by components (unrelated to the interface) in other parts of an application
  2. Acquired from data stored in a database that is accessible from the application
  3. Transmitted from systems external to the application in question

- In this phase of interface analysis, we examine the layout and visual appearance of the content displayed by the interface.
- Questions that are asked and answered include:
  - ***Are different types of data assigned to consistent geographic locations on the screen (e.g., photos always appear in the upper right-hand corner)?***
  - ***Can the user customize the screen location for content?***
  - ***Is proper on-screen identification assigned to all content?***
  - ***If a large report is to be presented, how should it be partitioned for ease of understanding?***
  - ***Will mechanisms be available for moving directly to summary information for large collections of data?***
  - ***Will graphical output be scaled to fit within the bounds of the display device that is used?***
  - ***How will color be used to enhance understanding?***
  - ***How will error messages and warnings be presented to the user?***

d. Analysis of the Work Environment

- Hackos and Redish discuss the importance of work environment analysis when they state:

> ***People do not perform their work in isolation. They are influenced by the activity around them, the physical characteristics of the workplace, the type of equipment they are using, and the work relationships they have with other people. If the products you design do not fit into the environment, they may be difficult or frustrating to use.***

- In certain applications, the user interface for a computer-based system is situated in a "user-friendly location," such as areas with
  - *proper lighting*
  - *suitable display height*
  - *easy keyboard access.*
- In other environments like factory floors or airplane cockpits, conditions may be less optimal. Lighting might be inadequate, noise could be a concern, and using a keyboard or mouse might not be feasible. Additionally, display placement may not be ideal in such scenarios.